Learning From Data

Mihir Rao

Contents

1 The Learning Problem **4**
1.1 Learning from examples . . . 4
1.2 Components of the learning problem . . . 4
1.3 The perceptron hypothesis set . . . 5
1.4 The perceptron learning algorithm . . . 5
1.5 Types of learning . . . 6
1.6 A learning puzzle . . . 7

2 Is Learning Feasible? **8**
2.1 Probability replaces certainty . . . 8
2.2 From marbles to prediction errors . . . 9
2.3 Choosing a hypothesis changes the question . . . 10
2.4 A guarantee for a finite hypothesis set . . . 11

3 The Linear Model I **12**
3.1 Input representation . . . 12
3.2 The pocket algorithm . . . 12
3.3 Linear regression . . . 13
3.4 Linear regression for classification . . . 14
3.5 Nonlinear transformation . . . 15

4 Error and Noise **16**
4.1 Nonlinear transformation, continued . . . 16
4.2 Error measures . . . 16
4.3 Choosing the error measure . . . 17
4.4 Noisy targets . . . 17
4.5 Preamble to the theory . . . 18

5 Training versus Testing **20**
5.1 From training to testing . . . 20
5.2 Dichotomies and the growth function . . . 20
5.3 Three illustrative examples . . . 21
5.4 Break points . . . 23

6 Theory of Generalization **24**
6.1 A combinatorial bound on the growth function . . . 24
6.2 Replacing M by the growth function . . . 26

7 The VC Dimension **28**
7.1 Definition . . . 28
7.2 VC dimension of perceptrons . . . 29
7.3 Interpreting the VC dimension . . . 30
7.4 Generalization bounds . . . 31

8 Bias-Variance Tradeoff **32**
8.1 Approximation versus generalization . . . 32
8.2 The bias-variance decomposition . . . 32
8.3 Example: a sine target . . . 33
8.4 Learning curves . . . 35
8.5 The linear regression case . . . 36

9 The Linear Model II **38**
9.1 Nonlinear transforms and their price . . . 38
9.2 The logistic regression model . . . 39
9.3 Cross-entropy error . . . 40
9.4 Gradient descent . . . 40

10 Neural Networks **42**
10.1 Stochastic gradient descent . . . 42
10.2 From perceptrons to multilayer networks . . . 43
10.3 The neural network model . . . 44
10.4 Backpropagation . . . 45

11 Overfitting **47**
11.1 What is overfitting? . . . 47
11.2 The role of noise . . . 48
11.3 Deterministic noise . . . 49
11.4 Dealing with overfitting . . . 50

12 Regularization **51**
12.1 A familiar example . . . 51
12.2 The polynomial model and weight constraints . . . 51
12.3 From constrained to augmented error . . . 52
12.4 Weight decay . . . 53
12.5 The general augmented error . . . 54
12.6 Choosing a regularizer . . . 54

13 Validation **56**
13.1 The validation set . . . 56
13.2 Why "validation" . . . 57
13.3 Model selection . . . 57
13.4 Cross validation . . . 58

14 Support Vector Machines **60**
14.1 Maximizing the margin . . . 60
14.2 The solution . . . 61
14.3 Nonlinear transforms . . . 63

15 Kernel Methods **64**
15.1 What the SVM needs from $\mathcal{Z}$ . . . 64
15.2 The kernel trick . . . 64
15.3 An infinite-dimensional $\mathcal{Z}$ . . . 65
15.4 Kernel formulation of the SVM . . . 65
15.5 Valid kernels . . . 66
15.6 Soft-margin SVM . . . 66
15.7 Two technical observations . . . 68

16 Radial Basis Functions **69**
16.1 The basic RBF model . . . 69

16.2 Relationship to nearest neighbors . . . 70
16.3 RBF with K centers . . . 70
16.4 RBF networks and neural networks . . . 71
16.5 RBF and kernel methods . . . 72
16.6 RBF and regularization . . . 73

17 Three Learning Principles **74**
17.1 Occam's razor . . . 74
17.2 Sampling bias . . . 75
17.3 Data snooping . . . 76

18 Epilogue **78**
18.1 The map of machine learning . . . 78
18.2 Bayesian learning . . . 78
18.3 Aggregation methods . . . 79

Lecture 1: The Learning Problem

1.1 Learning from examples

Machine learning infers a rule from observations. It is useful when there is structure to exploit, a satisfactory rule is unknown, and examples are available.

Example: *Movie recommendations.* Predict a viewer's rating by matching a vector of viewer preferences with a vector of movie attributes and summing their contributions. Comedy, action, and particular actors suggest possible factors, but the factors themselves can be learned from observed ratings. The data determine the representation as well as the predictions.

1.2 Components of the learning problem

For credit approval, an input describes an applicant, using attributes such as income and debt, and the output indicates whether the applicant is a good credit risk. The ideal decision rule is unknown; historical records provide examples of its behavior.

Definition 1.1 [Learning setup]. Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ the output space. An unknown target function $f : \mathcal{X} \to \mathcal{Y}$ supplies the labels in the training set

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}, \qquad y_n = f(\mathbf{x}_n).$$

A hypothesis set $\mathcal{H}$ contains the candidate functions $h : \mathcal{X} \to \mathcal{Y}$. A learning algorithm $\mathcal{A}$ uses $\mathcal{D}$ to choose a final hypothesis $g \in \mathcal{H}$ that should approximate f on new inputs.

The pair $(\mathcal{H}, \mathcal{A})$ is the learning model. The target and observations describe the problem; the hypothesis set and algorithm are the two components we choose to solve it.

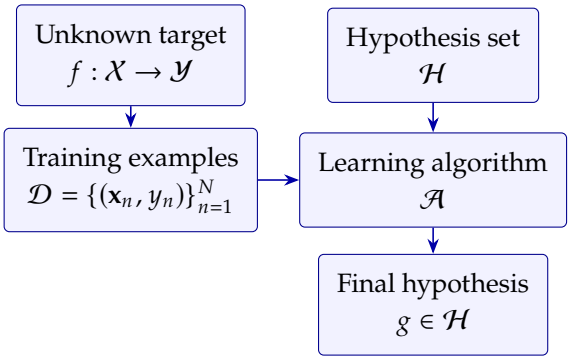


Figure 1: The target supplies training labels. The algorithm sees examples and candidate hypotheses, then returns g; it has no direct access to f.

1.3 The perceptron hypothesis set

For binary credit decisions, take $\mathcal{Y} = \{-1, +1\}$, with $+1$ representing approval. A PERCEPTRON compares a weighted sum of the d input attributes with a threshold τ:

$$h(\mathbf{x}) = \operatorname{sign}\left(\sum_{i=1}^{d} w_i x_i - \tau\right).$$

Absorb the threshold into a bias $w_0 = -\tau$ and append the constant coordinate $x_0 = 1$. From here on, $\mathbf{x} = (1, x_1, \ldots, x_d)^{\mathsf{T}}$ and $\mathbf{w} = (w_0, w_1, \ldots, w_d)^{\mathsf{T}}$, so

$$h(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}), \qquad \mathcal{H} = \{\mathbf{x} \mapsto \operatorname{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}) : \mathbf{w} \in \mathbb{R}^{d+1}\}.$$

In the original attribute space, the decision boundary is $w_0 + \sum_{i=1}^{d} w_i x_i = 0$: a line in two dimensions and a hyperplane in higher dimensions, provided $(w_1, \ldots, w_d) \neq 0$.

Definition: The training data are LINEARLY SEPARABLE if some $\mathbf{w}$ places every example strictly on its correct side: $y_n \mathbf{w}^{\mathsf{T}}\mathbf{x}_n > 0$ for all n.

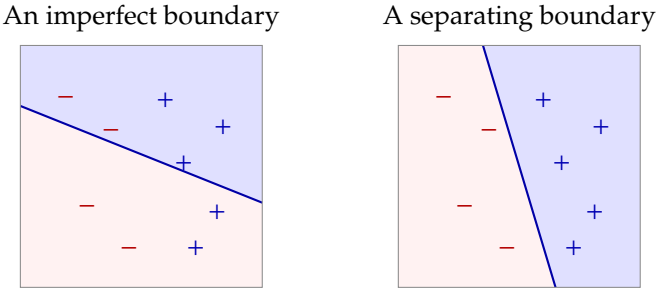


Figure 2: Schematic data with two candidate boundaries. The left boundary makes mistakes; the right one separates all training examples.

1.4 The perceptron learning algorithm

The PERCEPTRON LEARNING ALGORITHM (PLA) repeatedly chooses a misclassified training example $(\mathbf{x}_n, y_n)$ and updates

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n.}$$

For the procedure below, a zero score also triggers an update. This avoids assigning a boundary point to either class during training.

Algorithm 1.1: Perceptron learning algorithm

Input: Augmented examples $(\mathbf{x}_n, y_n)$, with $x_{n,0} = 1$ and $y_n \in \{-1, +1\}$
Output: A classifier that separates the training data, if the loop terminates

$\mathbf{w} \leftarrow \mathbf{0}$
while some n satisfies $y_n \mathbf{w}^\mathsf{T} \mathbf{x}_n \le 0$ **do**
    choose such an index n
    $\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$
return $g(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\mathsf{T} \mathbf{x})$

For a positive example, add $\mathbf{x}_n$; for a negative one, subtract it. The update moves the chosen example's score toward its correct sign:

$$y_n \mathbf{w}_{\text{new}}^\mathsf{T} \mathbf{x}_n = y_n \mathbf{w}_{\text{old}}^\mathsf{T} \mathbf{x}_n + \|\mathbf{x}_n\|^2.$$

This increases its signed score but need not correct it in one step, and other examples can become misclassified. The augmented update also adjusts the bias: $w_0 \leftarrow w_0 + y_n$.

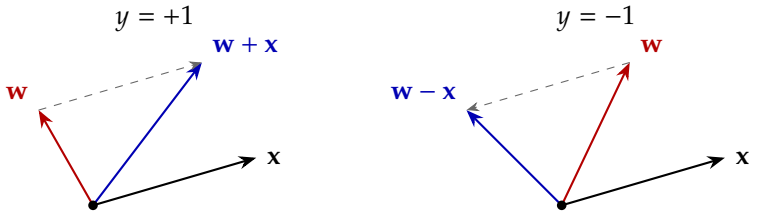


Figure 3: Schematic vector updates for misclassified examples. Dashed arrows complete the addition of $y\mathbf{x}$ to $\mathbf{w}$.

If the procedure stops, all training examples are correctly classified. Without a separating hyperplane, this stopping condition cannot be reached. Agreement on the training set still leaves the question of performance on new inputs.

1.5 Types of learning

The available feedback distinguishes three learning paradigms.

- Supervised learning receives input–label pairs $(\mathbf{x}, y)$. In coin recognition, mass and size are inputs, and the denomination is the label. Labeled examples determine decision regions for future coins.
- Unsupervised learning receives inputs without labels. The same coin measurements can reveal clusters, but their denominations are not supplied.
- Reinforcement learning receives feedback evaluating a chosen output or action, rather than the correct action itself. A backgammon player learns from game outcomes; a win or loss evaluates play without specifying the best move at every position.

1.6 A learning puzzle

Six labeled grids constrain an unknown binary rule. What label should be assigned to the new grid?

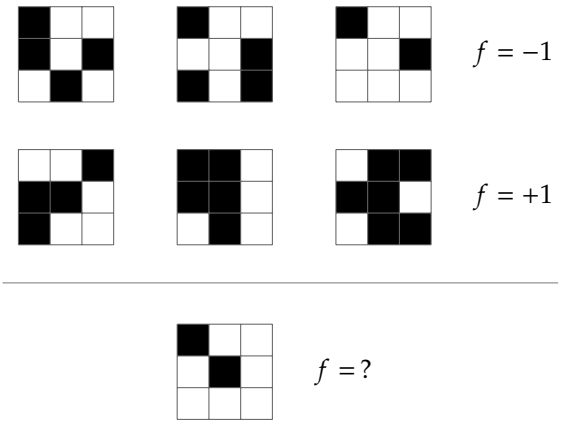


Figure 4: The six examples and unlabeled query in the learning puzzle. Black and white cells are the input attributes.

One consistent rule assigns +1 exactly when the center cell is black. It predicts +1 for the query. Another assigns +1 when the center is black *and* at least three cells are black. It fits the same six examples but predicts −1 for the query, which has only two black cells.

Thus, the examples alone do not uniquely determine the unseen label. Choosing a hypothesis set restricts which rules count as plausible; fitting the observations is only part of the learning problem.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 1: The Learning Problem, Caltech, April 3, 2012. Lecture slides.

Lecture 2: Is Learning Feasible?

Observations cannot determine an arbitrary target function at every unseen input. Two functions can agree on the entire training set and disagree elsewhere. A useful guarantee must instead relate performance on a random sample to performance on the population from which it was drawn.

2.1 Probability replaces certainty

Consider independent draws of red or green marbles, each red with an unknown probability μ. Let $Z_n = 1$ for a red draw and $Z_n = 0$ for a green one. The observed red fraction in N draws is

$$\nu = \frac{1}{N} \sum_{n=1}^{N} Z_n, \qquad \mathbb{E}[Z_n] = \mu.$$

An unrepresentative sample remains possible: a mostly red population can produce a mostly green sample. The question is how *probable* such a discrepancy is.

Theorem 2.1 [Hoeffding's inequality for a binary sample]. *For N independent Bernoulli observations with common mean μ and any $\epsilon > 0$,*

$$\mathbb{P}\big(|\nu - \mu| > \epsilon\big) \leq 2e^{-2N\epsilon^2}.$$

The bound holds for every $\mu \in [0, 1]$, without knowing its value. It decreases exponentially with N for a fixed tolerance ϵ; demanding a smaller tolerance requires a larger sample. A bound above one is valid but uninformative.

This is a probably approximately correct (PAC) statement: the estimate is within ϵ of the truth except on a small fraction of possible samples. Here μ is fixed and ν varies with the sample; the probability concerns the sampling procedure.

2.2 From marbles to prediction errors

Fix a target $f : \mathcal{X} \to \mathcal{Y}$ and a hypothesis $h : \mathcal{X} \to \mathcal{Y}$. Each marble represents an input $\mathbf{x} \in \mathcal{X}$: color it red when $h(\mathbf{x}) \neq f(\mathbf{x})$ and green when the two agree.

Draw $\mathbf{x}_1, \ldots, \mathbf{x}_N$ independently from a distribution P on $\mathcal{X}$, and set $y_n = f(\mathbf{x}_n)$. The distribution need not be known, but it must also describe the new inputs on which performance is measured.

Definition 2.2 [In-sample and out-of-sample error]. For classification, the in-sample error is the fraction of training examples misclassified by h:

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\{h(\mathbf{x}_n) \neq y_n\}.$$

The out-of-sample error is its probability of a mistake on a fresh input from P:

$$E_{\text{out}}(h) = \mathbb{P}_{\mathbf{x} \sim P}\big(h(\mathbf{x}) \neq f(\mathbf{x})\big).$$

The indicator $\mathbf{1}\{B\}$ equals one when the statement B holds and zero otherwise. In the marble analogy, $\nu = E_{\text{in}}(h)$ and $\mu = E_{\text{out}}(h)$.

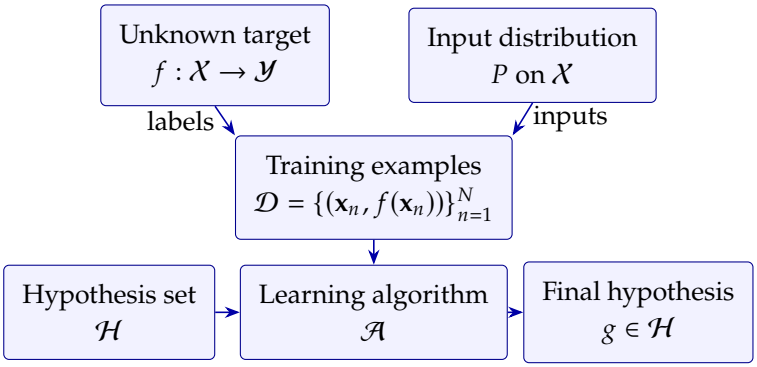


Figure 5: The distribution supplies training inputs and the target supplies their labels. The same P defines out-of-sample performance.

For a hypothesis fixed independently of the training sample, the error indicators are independent Bernoulli observations with common mean $E_{\text{out}}(h)$. Consequently,

$$\mathbb{P}_{\mathcal{D}}(|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon) \leq 2e^{-2N\epsilon^2}.$$

The subscript $\mathcal{D}$ emphasizes that this probability is over the random training set. A close estimate does not itself imply a small error: it may accurately reveal that h performs poorly.

2.3 Choosing a hypothesis changes the question

Learning searches a hypothesis set for a rule with small training error. The returned $g = \mathcal{A}(\mathcal{D})$ therefore depends on the same data used to compute $E_{\text{in}}(g)$. We cannot directly substitute this selected g for the fixed h in the preceding bound.

Example: *Searching for an unusual coin.* For one fair coin flipped ten times,

$$\mathbb{P}(\text{ten heads}) = 2^{-10} \approx 0.098\%.$$

For 1,000 fair coins with all flips independent, the probability that *some* coin produces ten heads is

$$1 - (1 - 2^{-10})^{1000} \approx 62.36\%.$$

Selecting an all-heads coin after inspecting the results does not make that coin biased. An event rare for one fixed choice can be common when there are many opportunities to find it.

For $\mathcal{H} = \{h_1, \ldots, h_M\}$, each hypothesis defines its own red/green coloring of the input space, with population red probability $E_{\text{out}}(h_m)$ and observed red fraction $E_{\text{in}}(h_m)$. The tempting choice is a hypothesis whose sample is mostly green, even if that sample is unusually favorable.

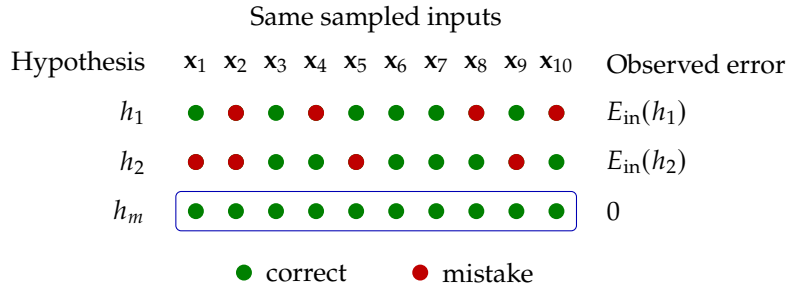


Figure 6: Different hypotheses color the same sampled inputs differently. An all-green row means zero training error, but does not establish zero population error. The colors illustrate the selection issue.

The hypotheses' sample errors need not be independent: they are computed on the same $\mathcal{D}$. Independence of the *training inputs* is what lets us apply Hoeffding to each fixed hypothesis separately.

2.4 A guarantee for a finite hypothesis set

Theorem 2.3 [Finite-hypothesis generalization bound]. *Let $\mathcal{H} = \{h_1, \dots, h_M\}$ be fixed before observing the data. For N independent training inputs from P, labeled by f, and any algorithm returning $g \in \mathcal{H}$,*

$$\mathbb{P}_{\mathcal{D}}\big(|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big) \le 2Me^{-2N\epsilon^2}.$$

Proof. Write $B_m = \{|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon\}$. If the selected g has a large discrepancy, then at least one candidate does. Hence

$$\begin{aligned}\mathbb{P}_{\mathcal{D}}\big(|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big) &\le \mathbb{P}_{\mathcal{D}}\left(\bigcup_{m=1}^{M} B_m\right) \\ &\le \sum_{m=1}^{M} \mathbb{P}_{\mathcal{D}}(B_m) \\ &\le \sum_{m=1}^{M} 2e^{-2N\epsilon^2} = 2Me^{-2N\epsilon^2}.\end{aligned}$$

The second step is the union bound, which requires no independence among the events B_m. The last step applies Hoeffding to each fixed h_m. □

The argument controls all candidates simultaneously, so it remains valid after selecting one using the data. For a desired failure probability $\delta \in (0, 1)$, solving $2Me^{-2N\epsilon^2} \le \delta$ gives

$$N \ge \frac{\log(2M/\delta)}{2\epsilon^2},$$

where log is the natural logarithm. With this many examples, with probability at least $1 - \delta$,

$$E_{\text{out}}(g) \le E_{\text{in}}(g) + \epsilon.$$

Learning therefore needs both a small training error and a reliable link between training and population errors. A larger hypothesis set offers more candidate rules, but increases the factor M in the guarantee; a larger sample can offset that cost. This argument establishes the link for a finite set, while leaving the algorithm responsible for finding a rule with small training error.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 2: Is Learning Feasible?, Caltech, April 5, 2012. Lecture slides.

Lecture 3: The Linear Model I

The signal $\mathbf{w}^\mathsf{T}\mathbf{x} = \sum_{i=0}^{d} w_i x_i$ underlies a family of linear models. Classification thresholds it; regression outputs it directly. Both are linear in $\mathbf{w}$, which makes a nonlinear transformation of the inputs possible without changing the algorithms.

3.1 Input representation

A data set of handwritten digits gives each image as a 16×16 grid of gray levels. The raw input is then

$$\mathbf{x} = (x_0, x_1, \ldots, x_{256}), \qquad \text{with weights } (w_0, w_1, \ldots, w_{256}),$$

so a linear model has 257 parameters. A feature extracts useful information from the raw input instead. For distinguishing the digits 1 and 5, two features suffice:

$$\mathbf{x} = (x_0, x_1, x_2), \qquad x_1 = \text{average intensity}, \quad x_2 = \text{symmetry},$$

with weights (w_0, w_1, w_2). A 1 uses less ink and is more symmetric than a 5, so in the (x_1, x_2) plane the two digits form largely separate clouds. The clouds overlap slightly, and the training data are not linearly separable.

3.2 The pocket algorithm

On these data, PLA never stops. Each update fixes one point but may break others, so E_{in} and E_{out} fluctuate erratically over the iterations, and the boundary at a forced stop depends on the arbitrary stopping time. It can be poor even when a good boundary was visited earlier.

The pocket algorithm runs PLA but keeps "in its pocket" the best weight vector seen so far, measured by E_{in}. Its in-sample error is therefore nonincreasing in the number of iterations, and on the digit data its final boundary separates the two classes better than the one where PLA happens to stop.

Algorithm 3.1: Pocket algorithm

Input: Examples $(\mathbf{x}_n, y_n)$ with $y_n \in \{-1, +1\}$, and an iteration count T
Output: The weights with the smallest in-sample error encountered

$\mathbf{w} \leftarrow \mathbf{0}$; $\hat{\mathbf{w}} \leftarrow \mathbf{w}$
for $t = 1, \dots, T$ **do**
  if some n satisfies $y_n \mathbf{w}^{\mathsf{T}} \mathbf{x}_n \le 0$ **then**
    $\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$ for such an n
  if $E_{\text{in}}(\mathbf{w}) < E_{\text{in}}(\hat{\mathbf{w}})$ **then**
    $\hat{\mathbf{w}} \leftarrow \mathbf{w}$
return $\hat{\mathbf{w}}$

3.3 Linear regression

Regression means real-valued output. In the credit example, classification decides approval, while regression sets the credit line, a dollar amount. The input $\mathbf{x}$ again lists applicant attributes (age, annual salary, years in residence, years in job, current debt, and so on), and the hypothesis is the signal itself:

$$h(\mathbf{x}) = \sum_{i=0}^{d} w_i x_i = \mathbf{w}^{\mathsf{T}} \mathbf{x}.$$

The data $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$ record credit lines $y_n \in \mathbb{R}$ chosen by credit officers, and linear regression tries to replicate those decisions. Quality is measured by squared error $(h(\mathbf{x}) - f(\mathbf{x}))^2$, so

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^{N} \big(h(\mathbf{x}_n) - y_n\big)^2.$$

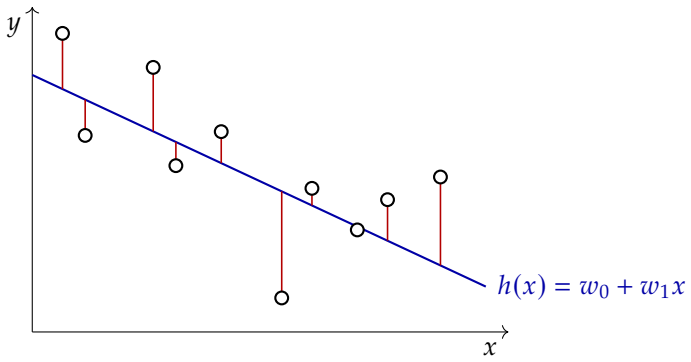


Figure 7: Schematic linear regression in one input variable. The vertical segments are the residuals whose squares E_{in} averages. With two inputs the line becomes a plane.

Matrix form. Stack the inputs as rows of the INPUT DATA MATRIX X and the targets into the TARGET VECTOR $\mathbf{y}$:

$$X = \begin{bmatrix} — \, \mathbf{x}_1^\mathsf{T} \, — \\ — \, \mathbf{x}_2^\mathsf{T} \, — \\ \vdots \\ — \, \mathbf{x}_N^\mathsf{T} \, — \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N.$$

Since the nth entry of $X\mathbf{w} - \mathbf{y}$ is $\mathbf{w}^\mathsf{T}\mathbf{x}_n - y_n$,

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{w}^\mathsf{T}\mathbf{x}_n - y_n)^2 = \frac{1}{N} \|X\mathbf{w} - \mathbf{y}\|^2.$$

Minimizing E_{in}. Expanding, $E_{\text{in}}(\mathbf{w}) = \frac{1}{N}\left(\mathbf{w}^\mathsf{T}X^\mathsf{T}X\mathbf{w} - 2\mathbf{w}^\mathsf{T}X^\mathsf{T}\mathbf{y} + \mathbf{y}^\mathsf{T}\mathbf{y}\right)$, a convex quadratic in $\mathbf{w}$, so any stationary point is a global minimum. Setting the gradient to zero,

$$\nabla E_{\text{in}}(\mathbf{w}) = \frac{2}{N} X^\mathsf{T}(X\mathbf{w} - \mathbf{y}) = \mathbf{0} \quad \Longleftrightarrow \quad X^\mathsf{T}X\mathbf{w} = X^\mathsf{T}\mathbf{y}.$$

When $X^\mathsf{T}X$ is invertible (that is, X has full column rank, as is typical when N is much larger than $d+1$),

$$\mathbf{w} = X^\dagger \mathbf{y}, \qquad X^\dagger = (X^\mathsf{T}X)^{-1}X^\mathsf{T}.$$

The matrix $X^\dagger$ is the PSEUDO-INVERSE of X; it satisfies $X^\dagger X = I$. Dimensionally, $X^\mathsf{T}X$ is a small $(d+1) \times (d+1)$ matrix, and $X^\dagger$ is $(d+1) \times N$.

Algorithm 3.2: Linear regression

Input: Examples $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$ with $x_{n,0} = 1$ and $y_n \in \mathbb{R}$
Output: Weights minimizing the in-sample squared error

1 Construct the input data matrix X and the target vector $\mathbf{y}$
2 Compute the pseudo-inverse $X^\dagger = (X^\mathsf{T}X)^{-1}X^\mathsf{T}$
3 **return** $\mathbf{w} = X^\dagger \mathbf{y}$

Unlike PLA, this is *one-step learning*: there is no iteration.

3.4 Linear regression for classification

Binary labels are also real numbers, $\pm 1 \in \mathbb{R}$. Linear regression on classification data finds $\mathbf{w}$ with $\mathbf{w}^\mathsf{T}\mathbf{x}_n \approx y_n = \pm 1$, in which case $\text{sign}(\mathbf{w}^\mathsf{T}\mathbf{x}_n)$ is likely to agree with y_n. Squared error is not the classification error, so this $\mathbf{w}$ is not necessarily the best classifier. It is, however, a good initial weight vector for PLA or the pocket algorithm. On the 1-versus-5 data, the regression boundary already separates most of the two classes.

3.5 Nonlinear transformation

Some data are not linearly separable at all. A training set may have one class near the center of the square $[-1, 1]^2$ and the other near the corners; a circle separates the data, but no line does.

The same issue arises in regression. The credit line depends on years in residence, but not linearly: indicator features such as $\mathbf{1}\{x_i < 1\}$ and $\mathbf{1}\{x_i > 5\}$ (a very short or a long residence) are more informative than x_i itself.

Both algorithms depend on linearity in the *weights*, not in the inputs. Linear regression implements $\sum_{i=0}^{d} w_i x_i$ and linear classification implements $\text{sign}\left(\sum_{i=0}^{d} w_i x_i\right)$; the x_i are just constants as far as the algorithms are concerned. Any fixed transformation of the inputs therefore preserves the machinery. For the circular data, use

$$\Phi : (x_1, x_2) \mapsto (x_1^2, x_2^2).$$

A circle $x_1^2 + x_2^2 = r^2$ in the original space becomes the line $z_1 + z_2 = r^2$ in the transformed space, where a linear classifier works.

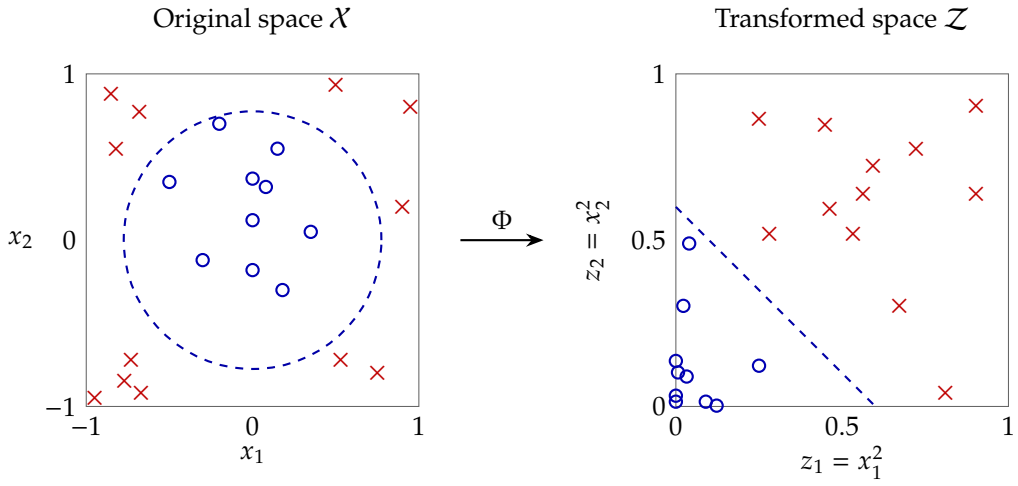


Figure 8: Schematic data with +1 near the center and −1 near the corners. Squaring each coordinate moves all points into $[0, 1]^2$, where a line separates the classes. The dashed curves show a circle and its image line.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 3: The Linear Model I, Caltech, April 10, 2012. Lecture slides.

Lecture 4: Error and Noise

4.1 Nonlinear transformation, continued

A transformation $\Phi : \mathcal{X} \to \mathcal{Z}$ turns a linear model in the feature space $\mathcal{Z}$ into a nonlinear model in $\mathcal{X}$. The procedure has four steps:

1. Start with the original data $\mathbf{x}_n \in \mathcal{X}$.
2. Transform them: $\mathbf{z}_n = \Phi(\mathbf{x}_n) \in \mathcal{Z}$.
3. Separate the data in $\mathcal{Z}$ with a linear classifier $\tilde{g}(\mathbf{z}) = \operatorname{sign}(\tilde{\mathbf{w}}^{\mathsf{T}}\mathbf{z})$.
4. Classify in $\mathcal{X}$ by $g(\mathbf{x}) = \tilde{g}(\Phi(\mathbf{x})) = \operatorname{sign}\big(\tilde{\mathbf{w}}^{\mathsf{T}}\Phi(\mathbf{x})\big)$.

For the circular data of Lecture 3, a line in $\mathcal{Z}$ pulls back to a closed curve around the center of $\mathcal{X}$. The objects transform as follows:

$$\begin{aligned} \mathbf{x} = (x_0, x_1, \dots, x_d) &\xrightarrow{\Phi} \mathbf{z} = (z_0, z_1, \dots, z_{\tilde{d}}), \\ \mathbf{x}_1, \dots, \mathbf{x}_N &\xrightarrow{\Phi} \mathbf{z}_1, \dots, \mathbf{z}_N, \\ y_1, \dots, y_N &\longrightarrow y_1, \dots, y_N \quad \text{(labels unchanged)}, \\ \text{no weights in } \mathcal{X} &\qquad \tilde{\mathbf{w}} = (w_0, w_1, \dots, w_{\tilde{d}}). \end{aligned}$$

The dimension $\tilde{d}$ of $\mathcal{Z}$ may differ from d. Learning happens entirely in $\mathcal{Z}$; the final hypothesis is still a function of $\mathbf{x}$.

4.2 Error measures

To make "$h \approx f$" precise, an error measure $E(h, f)$ quantifies how far h is from f. Almost always it is built from a pointwise error $\mathrm{e}\big(h(\mathbf{x}), f(\mathbf{x})\big)$:

$$\begin{aligned} &\text{squared error:} \quad \mathrm{e}\big(h(\mathbf{x}), f(\mathbf{x})\big) = \big(h(\mathbf{x}) - f(\mathbf{x})\big)^2, \\ &\text{binary error:} \quad \mathrm{e}\big(h(\mathbf{x}), f(\mathbf{x})\big) = \mathbf{1}\{h(\mathbf{x}) \neq f(\mathbf{x})\}. \end{aligned}$$

The overall error averages pointwise errors, over the sample or over the input distribution:

$$E_{\text{in}}(h) = \frac{1}{N}\sum_{n=1}^{N} \mathrm{e}\big(h(\mathbf{x}_n), f(\mathbf{x}_n)\big), \qquad E_{\text{out}}(h) = \mathbb{E}_{\mathbf{x}}\Big[\mathrm{e}\big(h(\mathbf{x}), f(\mathbf{x})\big)\Big].$$

With binary error these reduce to the definitions of Lecture 2.

4.3 Choosing the error measure

In fingerprint verification, $f(\mathbf{x}) = +1$ for an authorized person and -1 for an intruder. A classifier can make two kinds of mistake: a FALSE ACCEPT ($h = +1$, $f = -1$) and a FALSE REJECT ($h = -1$, $f = +1$). The application determines how to penalize each.

Mistake types

		$f = +1$	$f = -1$
h	$+1$	none	FA
	-1	FR	none

Supermarket

		$f = +1$	$f = -1$
h	$+1$	0	1
	-1	10	0

CIA

		$f = +1$	$f = -1$
h	$+1$	0	1000
	-1	1	0

Table 1: Penalty matrices for fingerprint verification. FA is a false accept and FR a false reject.

- *Supermarket discounts.* A false reject is costly: the customer gets annoyed. A false accept only gives away a discount, and the intruder has left a fingerprint.
- *CIA security.* A false accept is a disaster. A false reject can be tolerated: an employee can simply try again.

The same verification problem thus calls for different error measures and leads to different final hypotheses.

The error measure should be *specified by the user*. When that is not possible, there are two alternatives:

- *plausible* measures with an analytic justification, such as squared error, which corresponds to Gaussian noise;
- *friendly* measures that make the optimization easy, such as a closed-form solution or a convex objective.

The error measure enters learning in two places: it defines how $g \approx f$ is judged, and the learning algorithm uses it to evaluate candidate hypotheses on the training data.

4.4 Noisy targets

The "target function" need not be a function. Two credit applicants with identical records (age, salary, years in residence, years in job, debt, and so on) can behave differently. Instead of $y = f(\mathbf{x})$, use a TARGET DISTRIBUTION $P(y \mid \mathbf{x})$. Each example $(\mathbf{x}, y)$ is then generated by the joint distribution

$$P(\mathbf{x}, y) = P(\mathbf{x})\, P(y \mid \mathbf{x}).$$

A noisy target is a deterministic target plus noise:

$$y = f(\mathbf{x}) + \big(y - f(\mathbf{x})\big), \qquad f(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}].$$

A deterministic target is the special case in which $P(y \mid \mathbf{x})$ puts all its mass on $y = f(\mathbf{x})$. With noise, the errors compare h with the observed y:

$$E_{\text{in}}(h) = \frac{1}{N}\sum_{n=1}^{N} \mathrm{e}\big(h(\mathbf{x}_n), y_n\big), \qquad E_{\text{out}}(h) = \mathbb{E}_{\mathbf{x},y}\Big[\mathrm{e}\big(h(\mathbf{x}), y\big)\Big].$$

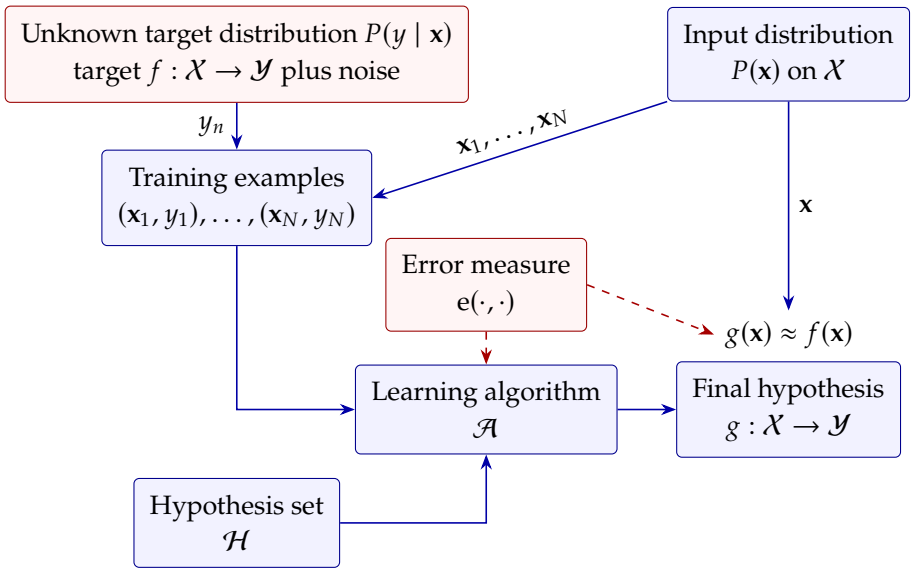


Figure 9: The learning diagram with a noisy target and an error measure. The input distribution supplies training inputs and the test point $\mathbf{x}$; the error measure judges $g(\mathbf{x}) \approx f(\mathbf{x})$ and guides the algorithm.

Both $P(y \mid \mathbf{x})$ and $P(\mathbf{x})$ describe probabilistic aspects of $\mathbf{x}$ and y, but they play different roles. The target distribution $P(y \mid \mathbf{x})$ is what we are trying to learn. The input distribution $P(\mathbf{x})$ only quantifies the relative importance of different $\mathbf{x}$. Merging them into $P(\mathbf{x}, y)$ mixes the two concepts.

4.5 Preamble to the theory

Lecture 2 showed that learning is feasible in the sense that, with high probability,

$$E_{\text{out}}(g) \approx E_{\text{in}}(g).$$

This alone is not learning. We need $g \approx f$, which means $E_{\text{out}}(g) \approx 0$. That goal splits into two parts:

$$E_{\text{out}}(g) \approx 0 \quad \text{follows from} \quad \underbrace{E_{\text{out}}(g) \approx E_{\text{in}}(g)}_{\text{Lecture 2}} \quad \text{and} \quad \underbrace{E_{\text{in}}(g) \approx 0}_{\text{Lecture 3}}.$$

Learning is thus split into two questions:

1. Can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?
2. Can we make $E_{\text{in}}(g)$ small enough?

The theory to come characterizes the feasibility of learning for infinite M, and the tradeoff between the two questions. A more complex model lowers E_{in} but widens the gap $E_{\text{out}} - E_{\text{in}}$. Model complexity will be measured by the VC DIMENSION d_{VC}, and the out-of-sample error is smallest at an intermediate value d^*_{VC}.

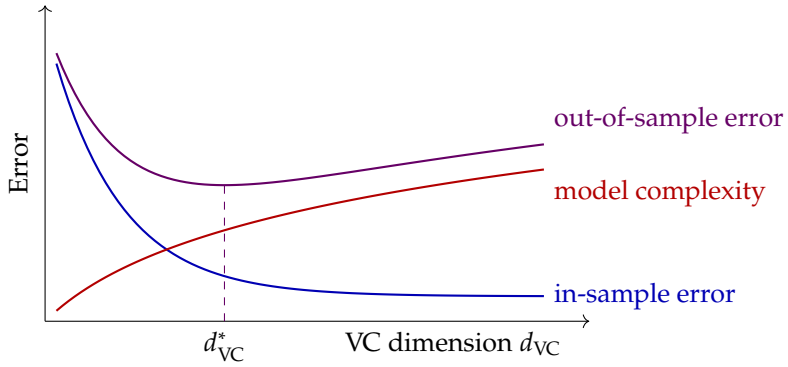


Figure 10: Schematic shape of the tradeoff, not computed from data. In-sample error falls as d_{VC} grows while the model-complexity term rises; out-of-sample error is smallest at d^*_{VC}.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 4: Error and Noise, Caltech, April 12, 2012. Lecture slides.

Lecture 5: Training versus Testing

5.1 From training to testing

Compare two settings for the same Hoeffding-type bound. In *testing*, as in a final exam, a single hypothesis is fixed before the data are seen:

$$\mathbb{P}\big[|E_{\text{in}} - E_{\text{out}}| > \epsilon\big] \le 2e^{-2\epsilon^2 N}.$$

In *training*, the hypothesis is chosen from M candidates using the data:

$$\mathbb{P}\big[|E_{\text{in}} - E_{\text{out}}| > \epsilon\big] \le 2Me^{-2\epsilon^2 N}.$$

For infinite hypothesis sets such as perceptrons, $M = \infty$ and the training bound is useless. The goal is to replace M by something finite.

Where the factor M comes from. Call $\mathcal{B}_m = \big\{|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon\big\}$ the BAD EVENT for h_m. The union bound gives

$$\mathbb{P}[\mathcal{B}_1 \text{ or } \mathcal{B}_2 \text{ or } \cdots \text{ or } \mathcal{B}_M] \le \mathbb{P}[\mathcal{B}_1] + \mathbb{P}[\mathcal{B}_2] + \cdots + \mathbb{P}[\mathcal{B}_M],$$

which is tight only when the bad events do not overlap. It charges the full M terms in the worst case.

Bad events overlap heavily. Take two perceptrons h_1, h_2 whose boundaries differ by a slight rotation. Their E_{out} values differ only by the probability of the thin region where they disagree, and their E_{in} values differ only through the few data points, if any, that fall in that region. Hence

$$|E_{\text{in}}(h_1) - E_{\text{out}}(h_1)| \approx |E_{\text{in}}(h_2) - E_{\text{out}}(h_2)|,$$

so $\mathcal{B}_1$ and $\mathcal{B}_2$ nearly coincide, and the union bound greatly overcounts.

5.2 Dichotomies and the growth function

Instead of the whole input space, consider only a finite set of input points and count the distinct ways a hypothesis set can label them.

Definition 5.1 [Dichotomy]. A hypothesis is a function $h : \mathcal{X} \to \{-1, +1\}$. A DICHOTOMY is its restriction to N points, $h : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \to \{-1, +1\}$. Write $\mathcal{H}(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ for the set of dichotomies

that $\mathcal{H}$ generates on these points.

The number of hypotheses $|\mathcal{H}|$ can be infinite, but the number of dichotomies is finite:

$$|\mathcal{H}(\mathbf{x}_1, \ldots, \mathbf{x}_N)| \leq 2^N.$$

This count is a candidate for replacing M, since hypotheses with the same dichotomy have the same E_{in}.

Definition 5.2 [Growth function]. The growth function counts the most dichotomies on any N points:

$$m_{\mathcal{H}}(N) = \max_{\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathcal{X}} |\mathcal{H}(\mathbf{x}_1, \ldots, \mathbf{x}_N)|.$$

It satisfies $m_{\mathcal{H}}(N) \leq 2^N$. If $\mathcal{H}$ generates all 2^N dichotomies on some N points, it shatters them.

Perceptrons in the plane. Three points not on a line can be labeled in all 8 ways by a line, so $m_{\mathcal{H}}(3) = 8$. Three collinear points allow fewer (the middle point cannot differ from both ends), but the maximum is what counts. For four points, $m_{\mathcal{H}}(4) = 14$: some two of the 16 dichotomies are always impossible. For four points in convex position, these are the two labelings that alternate around the quadrilateral, which would need the two diagonals on opposite sides of one line. If one point lies inside the triangle of the other three, the impossible labelings give that point the sign opposite to all three others, since a half-plane containing the three vertices contains the triangle. Configurations with three or more collinear points allow fewer still.

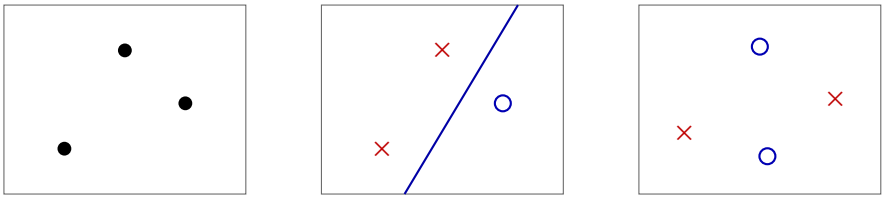


Figure 11: Three points in general position are shattered by lines; the middle panel shows one of the eight dichotomies. The alternating labeling of four points in convex position cannot be produced by any line.

5.3 Three illustrative examples

Positive rays. $\mathcal{H}$ consists of $h : \mathbb{R} \to \{-1, +1\}$ with $h(x) = \text{sign}(x - a)$: -1 to the left of the threshold a and $+1$ to the right. On N distinct points, the dichotomy depends only on which of the $N + 1$ gaps between and around the sorted points contains a, and each gap gives a different dichotomy. Hence

$$m_{\mathcal{H}}(N) = N + 1.$$

Positive intervals. Now $h(x) = +1$ inside an interval and -1 outside. A dichotomy is fixed by the two gaps, among the $N+1$, containing the interval's ends. Two different gaps give $\binom{N+1}{2}$ distinct dichotomies with a nonempty block of $+1$ points; both ends in the same gap give the all-(-1) dichotomy. Therefore

$$m_{\mathcal{H}}(N) = \binom{N+1}{2} + 1 = \tfrac{1}{2}N^2 + \tfrac{1}{2}N + 1.$$

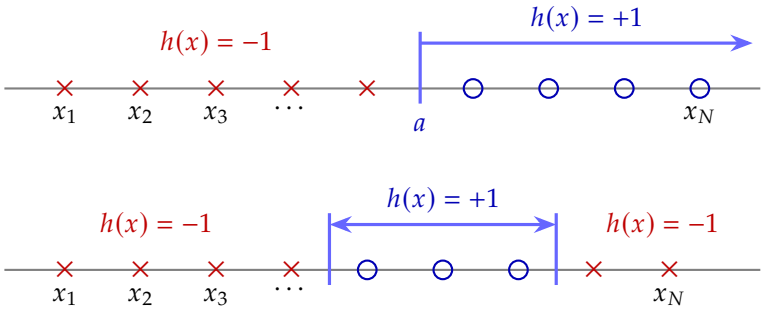


Figure 12: Positive rays (top) and positive intervals (bottom). Circles are points labeled $+1$ and crosses points labeled -1.

Convex sets. $\mathcal{H}$ consists of $h : \mathbb{R}^2 \to \{-1, +1\}$ such that $\{\mathbf{x} : h(\mathbf{x}) = +1\}$ is convex. Place the N points on a circle. For any dichotomy, take the convex hull of the points labeled $+1$ (the empty set if there are none). Every point on a circle is an extreme point of any polygon inscribed in it, so the hull contains no point labeled -1. All 2^N dichotomies are realized, the N points are shattered, and

$$m_{\mathcal{H}}(N) = 2^N.$$

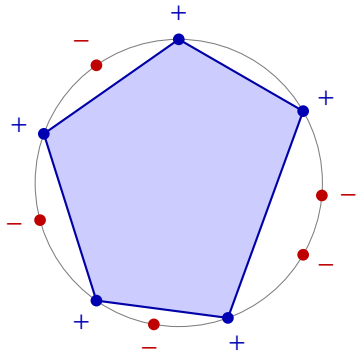


Figure 13: Points on a circle are shattered by convex sets: the convex hull of the $+1$ points excludes every -1 point.

Back to the big picture. If $m_{\mathcal{H}}(N)$ could replace M, the bound would read $2\, m_{\mathcal{H}}(N)\, e^{-2\epsilon^2 N}$. When $m_{\mathcal{H}}(N)$ is a polynomial in N, the exponential factor eventually dominates, and the bound goes to zero as N grows. This holds for positive rays and positive intervals, but not for convex sets.

Two things remain: to justify the replacement, and to show that $m_{\mathcal{H}}(N)$ is polynomial without computing it exactly.

5.4 Break points

Definition 5.3 [Break point]. If no data set of size k can be shattered by $\mathcal{H}$, then k is a **break point** for $\mathcal{H}$. Equivalently, $m_{\mathcal{H}}(k) < 2^k$.

If k is a break point, so is every larger number: shattering $k+1$ points would shatter any k of them. For 2D perceptrons, $k = 4$ is a break point, since $m_{\mathcal{H}}(4) = 14 < 16$. For the examples above:

- positive rays have break point $k = 2$, since no ray labels the left point $+1$ and the right point -1;
- positive intervals have break point $k = 3$, since no interval labels three points $+1, -1, +1$ from left to right;
- convex sets have no break point ("$k = \infty$").

Indeed, $N + 1 < 2^N$ first at $N = 2$, and $\frac{1}{2}N^2 + \frac{1}{2}N + 1 < 2^N$ first at $N = 3$.

Theorem 5.4 [Main result]. *If $\mathcal{H}$ has no break point, then $m_{\mathcal{H}}(N) = 2^N$ for all N. If $\mathcal{H}$ has any break point, then $m_{\mathcal{H}}(N)$ is bounded by a polynomial in N.*

The first statement is the definition. The second is proved in Lecture 6: a single size at which shattering fails forces polynomial growth.

Example: *A puzzle.* Suppose $\mathcal{H}$ has break point $k = 2$, so no two of the points $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ may be shattered. How many dichotomies can $\mathcal{H}$ have on the three points? Writing $\circ$ for -1 and $\bullet$ for $+1$, the following four are compatible:

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$
$\circ$	$\circ$	$\circ$
$\circ$	$\circ$	$\bullet$
$\circ$	$\bullet$	$\circ$
$\bullet$	$\circ$	$\circ$

Each of the four remaining patterns would complete all four labelings of some pair: $\circ\bullet\bullet$ on $(\mathbf{x}_2, \mathbf{x}_3)$, $\bullet\circ\bullet$ on $(\mathbf{x}_1, \mathbf{x}_3)$, $\bullet\bullet\circ$ and $\bullet\bullet\bullet$ on $(\mathbf{x}_1, \mathbf{x}_2)$. So the list cannot be extended.

In fact no five dichotomies on three points avoid shattering a pair. The projection onto $(\mathbf{x}_1, \mathbf{x}_2)$ takes at most three values, so two different values $u \neq v$ each occur with both labels of $\mathbf{x}_3$. If u and v differ on $\mathbf{x}_1$, the pair $(\mathbf{x}_1, \mathbf{x}_3)$ is shattered; otherwise they differ on $\mathbf{x}_2$ and $(\mathbf{x}_2, \mathbf{x}_3)$ is shattered. Thus break point 2 allows at most 4 of the 8 dichotomies on three points.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 5: Training versus Testing, Caltech, April 17, 2012. Lecture slides.

Lecture 6: Theory of Generalization

Recall from Lecture 5 that a dichotomy is the pattern of ±1 labels a hypothesis produces on a set of inputs, and that the growth function

$$m_{\mathcal{H}}(N) = \max_{\mathbf{x}_1,\dots,\mathbf{x}_N \in \mathcal{X}} \left|\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N)\right|$$

counts the most dichotomies $\mathcal{H}$ can produce on N points. A break point k is a size at which no set of k points is shattered, so $m_{\mathcal{H}}(k) < 2^k$. This lecture proves two facts: a break point makes $m_{\mathcal{H}}(N)$ polynomial in N, and $m_{\mathcal{H}}$ can replace the number of hypotheses M in the finite-set generalization bound.

6.1 A combinatorial bound on the growth function

To bound $m_{\mathcal{H}}(N)$ without reference to any particular $\mathcal{H}$, consider only the break-point constraint.

Definition 6.1 [The quantity $B(N, k)$]. $B(N, k)$ is the maximum number of distinct dichotomies on N points such that no subset of k of the points is shattered by them.

If $\mathcal{H}$ has break point k, its dichotomies on any N points satisfy this constraint, so $m_{\mathcal{H}}(N) \le B(N, k)$. The aim is $m_{\mathcal{H}}(N) \le B(N, k) \le$ a polynomial in N.

Two boundary cases are immediate. For $k = 1$, no single point may take both labels, so every point has a fixed label and $B(N, 1) = 1$. For one point and $k \ge 2$, both labels are allowed and nothing larger can be shattered, so $B(1, k) = 2$.

Lemma 6.2 [Recursive bound]. *For $N \ge 2$ and $k \ge 2$,*

$$B(N, k) \le B(N-1, k) + B(N-1, k-1).$$

Proof. List a maximal collection of $B(N, k)$ dichotomies as rows of a table with columns $\mathbf{x}_1, \dots, \mathbf{x}_N$, and group the rows by their restriction to $\mathbf{x}_1, \dots, \mathbf{x}_{N-1}$.

- S_1: the α rows whose restriction appears only once.
- $S_2 = S_2^+ \cup S_2^-$: rows whose restriction appears twice, once with $\mathbf{x}_N = +1$ (in S_2^+) and once with $\mathbf{x}_N = -1$ (in S_2^-). Each part has β rows.

Thus $B(N, k) = \alpha + 2\beta$.
Bounding $\alpha + \beta$. Delete the column $\mathbf{x}_N$. The rows of S_1 and S_2^+ remain distinct on the first $N - 1$ points, and they shatter no k of those points because the original rows did not. Hence

$\alpha + \beta \le B(N-1,k)$.
Bounding β. The β restrictions of S_2^+ (the same as those of S_2^-) cannot shatter any $k-1$ of the first $N-1$ points. Otherwise, since each of those restrictions occurs with both values of $\mathbf{x}_N$, the rows of S_2 would shatter those $k-1$ points together with $\mathbf{x}_N$, which is k points. Hence $\beta \le B(N-1,k-1)$.
Adding the two bounds gives $B(N,k) = (\alpha+\beta)+\beta \le B(N-1,k) + B(N-1,k-1)$. □

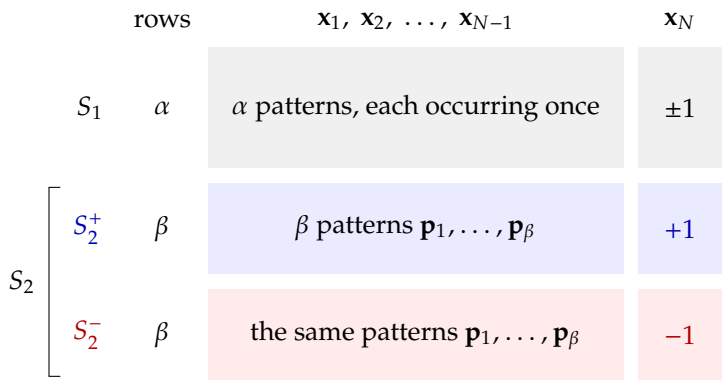


Figure 14: Rows of a maximal table grouped by their pattern on the first $N-1$ points. Each pattern in S_2 occurs with both labels on $\mathbf{x}_N$.

The boundary values and the recursion generate upper bounds on $B(N,k)$ row by row. Each entry is at most the sum of the entry above it and the entry above and to its left. For example, $B(3,2) \le B(2,2) + B(2,1) = 3+1 = 4$.

$N \setminus k$	1	2	3	4	5	6
1	1	2	2	2	2	2
2	1	3	4	4	4	4
3	1	4	7	8	8	8
4	1	5	11	15	16	16
5	1	6	16	26	31	32
6	1	7	22	42	57	63

Table 2: Upper bounds on $B(N,k)$ generated by the recursion.

Theorem 6.3 [Polynomial bound on $B(N,k)$]. *For all $N \ge 1$ and $k \ge 1$,*

$$B(N,k) \le \sum_{i=0}^{k-1} \binom{N}{i}.$$

Proof. Induct on N, for all k simultaneously. The boundary cases hold: for $k = 1$ the sum is $\binom{N}{0} = 1$, and for $N = 1$, $k \geq 2$ it is $\binom{1}{0} + \binom{1}{1} = 2$. For the induction step, apply Lemma 6.2 and the hypothesis for $N - 1$:

$$B(N,k) \leq \sum_{i=0}^{k-1} \binom{N-1}{i} + \sum_{i=0}^{k-2} \binom{N-1}{i} = 1 + \sum_{i=1}^{k-1} \left[\binom{N-1}{i} + \binom{N-1}{i-1} \right]$$
$$= 1 + \sum_{i=1}^{k-1} \binom{N}{i} = \sum_{i=0}^{k-1} \binom{N}{i}.$$

The second sum was reindexed by $i \mapsto i - 1$, and Pascal's rule combines each bracketed pair into $\binom{N}{i}$. □

Corollary 6.4 [The growth function is polynomial]**.** *If $\mathcal{H}$ has a break point k, then*

$$m_{\mathcal{H}}(N) \leq \sum_{i=0}^{k-1} \binom{N}{i},$$

a polynomial in N of degree $k - 1$.

For a given $\mathcal{H}$ the break point is a fixed number, so the degree does not grow with N. The bound is tight for two earlier examples:

$\mathcal{H}$	Break point	$m_{\mathcal{H}}(N)$	$\sum_{i=0}^{k-1} \binom{N}{i}$
Positive rays	$k = 2$	$N + 1$	$N + 1$
Positive intervals	$k = 3$	$\frac{1}{2}N^2 + \frac{1}{2}N + 1$	$\frac{1}{2}N^2 + \frac{1}{2}N + 1$
2D perceptrons	$k = 4$	not computed	$\frac{1}{6}N^3 + \frac{5}{6}N + 1$

For perceptrons, the polynomial bound holds without computing the growth function exactly.

6.2 Replacing M by the growth function

For a finite hypothesis set, Lecture 2 gave $\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2Me^{-2\epsilon^2 N}$. The goal is to replace M, which is infinite for perceptrons, by $m_{\mathcal{H}}(N)$. The argument below is a sketch of the proof.

Overlaps among bad events. View each data set $\mathcal{D}$ as a point in the space of all data sets, with total probability one. For a fixed h, Hoeffding says that the bad data sets, where $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$, have small probability. The union bound adds these probabilities over all hypotheses, as if the bad regions were disjoint. In fact they overlap heavily: hypotheses that produce the same dichotomy on

$\mathcal{D}$ have the same E_{in}, so on that sample they behave as one hypothesis. The growth function counts how many genuinely different behaviors a sample can reveal.

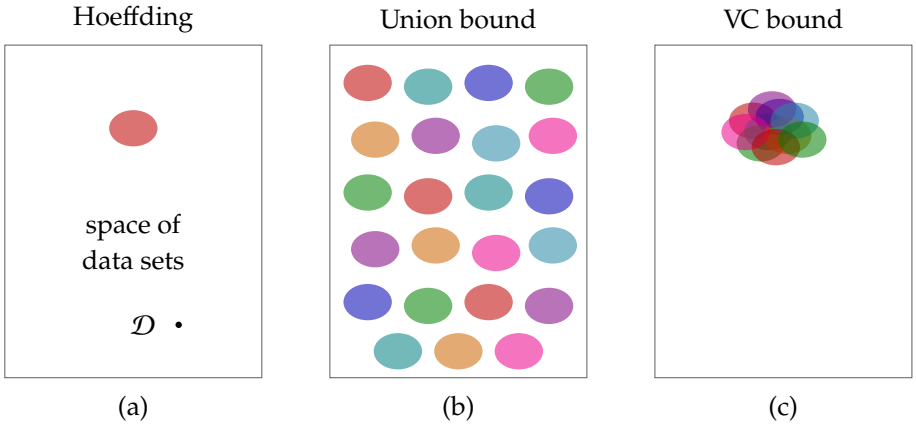


Figure 15: Schematic of bad regions in the space of data sets. (a) One hypothesis. (b) The union bound treats the regions for different hypotheses as disjoint. (c) The VC bound exploits their overlap.

Handling E_{out}. The dichotomy argument applies to E_{in}, which depends only on the sample, but $E_{\text{out}}(h)$ depends on h over the whole input space. Draw a second independent sample $\mathcal{D}'$ of size N, with in-sample error $E'_{\text{in}}(h)$. Both $E_{\text{in}}(h)$ and $E'_{\text{in}}(h)$ track $E_{\text{out}}(h)$, so if E_{in} deviates from E_{out}, it is likely to deviate from E'_{in} as well. The discrepancy $|E_{\text{in}}(h) - E'_{\text{in}}(h)|$ depends only on the dichotomy h produces on the $2N$ points of $\mathcal{D} \cup \mathcal{D}'$, and there are at most $m_{\mathcal{H}}(2N)$ of those.

Putting it together. The result is not quite $2\, m_{\mathcal{H}}(N) e^{-2\epsilon^2 N}$; the two-sample step costs constants and doubles the sample size in the growth function.

Theorem 6.5 [The Vapnik–Chervonenkis inequality]. *For any hypothesis set $\mathcal{H}$ and any $g \in \mathcal{H}$ chosen from the data,*

$$\mathbb{P}\big[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big] \le 4\, m_{\mathcal{H}}(2N)\, e^{-\frac{1}{8}\epsilon^2 N}.$$

If $\mathcal{H}$ has a break point, $m_{\mathcal{H}}(2N)$ is bounded by a polynomial in N. The exponential factor then dominates, and the right side tends to zero as $N \to \infty$ for every fixed $\epsilon > 0$.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 6: Theory of Generalization, Caltech, April 19, 2012. Lecture slides.

Lecture 7: The VC Dimension

Lecture 6 showed that a break point k gives $m_{\mathcal{H}}(N) \le \sum_{i=0}^{k-1} \binom{N}{i}$, and that the growth function can replace M in the generalization bound:

$$\mathbb{P}\big[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big] \le 4\, m_{\mathcal{H}}(2N)\, e^{-\frac{1}{8}\epsilon^2 N}.$$

The VC dimension summarizes the break point as a single number attached to $\mathcal{H}$.

7.1 Definition

Definition 7.1 [VC dimension]. The VC dimension of a hypothesis set $\mathcal{H}$, denoted $d_{\text{VC}}(\mathcal{H})$, is the largest value of N for which $m_{\mathcal{H}}(N) = 2^N$: the most points $\mathcal{H}$ can shatter. If $\mathcal{H}$ shatters sets of every size, $d_{\text{VC}}(\mathcal{H}) = \infty$.

Two consequences follow directly:

- if $N \le d_{\text{VC}}(\mathcal{H})$, then $\mathcal{H}$ can shatter some set of N points;
- if $k > d_{\text{VC}}(\mathcal{H})$, then k is a break point for $\mathcal{H}$.

The smallest break point is $k = d_{\text{VC}} + 1$. Substituting it into the bound from Lecture 6 expresses the growth function in terms of the VC dimension:

$$m_{\mathcal{H}}(N) \le \sum_{i=0}^{d_{\text{VC}}} \binom{N}{i},$$

a polynomial whose maximum power is $N^{d_{\text{VC}}}$.

Example: *Three hypothesis sets.* Positive rays have $d_{\text{VC}} = 1$, and 2D perceptrons have $d_{\text{VC}} = 3$: three points in general position can be shattered, but no four points can. Convex sets in the plane have $d_{\text{VC}} = \infty$. Place N points on a circle; for any choice of labels, the convex hull of the +1 points contains none of the others, so every dichotomy is realized.

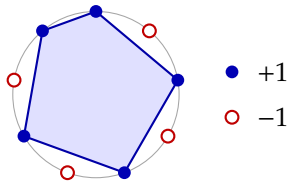


Figure 16: Convex sets shatter points on a circle: the hull of the +1 points (filled) excludes the −1 points.

Note. A finite VC dimension implies that the final hypothesis $g \in \mathcal{H}$ will generalize, given enough data. The guarantee is independent of the learning algorithm, the input distribution, and the target function. Only the hypothesis set and the training examples enter it.

7.2 VC dimension of perceptrons

For $d = 2$, $d_{\text{VC}} = 3$. The general answer counts the bias coordinate.

Theorem 7.2 [VC dimension of the perceptron]. *Perceptrons in* $\mathbb{R}^d$, $h(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\mathsf{T}\mathbf{x})$ *with* $\mathbf{x} = (1, x_1, \ldots, x_d)^\mathsf{T}$ *and* $\mathbf{w} \in \mathbb{R}^{d+1}$, *have* $d_{\text{VC}} = d + 1$.

The proof has two directions. For $d_{\text{VC}} \geq d + 1$ it suffices to exhibit one set of $d + 1$ points that is shattered. For $d_{\text{VC}} \leq d + 1$, a single unshatterable set is not enough: we must show that *no* set of $d + 2$ points can be shattered.

Proof. Lower bound. Take the origin and the d standard basis vectors of $\mathbb{R}^d$. In augmented coordinates they form the rows of

$$X = \begin{bmatrix} — \, \mathbf{x}_1^\mathsf{T} \, — \\ — \, \mathbf{x}_2^\mathsf{T} \, — \\ \vdots \\ — \, \mathbf{x}_{d+1}^\mathsf{T} \, — \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ 1 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ 1 & 0 & \cdots & 0 & 1 \end{bmatrix},$$

which is invertible. For any labels $\mathbf{y} = (y_1, \ldots, y_{d+1})^\mathsf{T} \in \{-1, +1\}^{d+1}$, choose $\mathbf{w} = X^{-1}\mathbf{y}$. Then $X\mathbf{w} = \mathbf{y}$, so $\operatorname{sign}(X\mathbf{w}) = \mathbf{y}$. Every dichotomy is realized, and $d_{\text{VC}} \geq d + 1$.
Upper bound. Take any $d + 2$ points $\mathbf{x}_1, \ldots, \mathbf{x}_{d+2} \in \mathbb{R}^{d+1}$ (augmented). There are more vectors than dimensions, so they are linearly dependent, and some $\mathbf{x}_j$ is a combination of the others:

$$\mathbf{x}_j = \sum_{i \neq j} a_i \mathbf{x}_i.$$

Not all a_i are zero, since $\mathbf{x}_j$ has first coordinate 1. Consider the dichotomy in which each $\mathbf{x}_i$ with $a_i \neq 0$ gets $y_i = \operatorname{sign}(a_i)$ and $\mathbf{x}_j$ gets $y_j = -1$ (other points arbitrary). Suppose a perceptron $\mathbf{w}$ produced it. For every i with $a_i \neq 0$, $\operatorname{sign}(\mathbf{w}^\mathsf{T}\mathbf{x}_i) = \operatorname{sign}(a_i)$, so $a_i \mathbf{w}^\mathsf{T}\mathbf{x}_i > 0$. Therefore

$$\mathbf{w}^\mathsf{T}\mathbf{x}_j = \sum_{i \neq j} a_i \, \mathbf{w}^\mathsf{T}\mathbf{x}_i > 0,$$

and $y_j = \operatorname{sign}(\mathbf{w}^\mathsf{T}\mathbf{x}_j) = +1$, a contradiction. No set of $d+2$ points is shattered, so $d_{\text{VC}} \leq d+1$. □

The number $d + 1$ is exactly the number of perceptron parameters $w_0, w_1, \ldots, w_d$.

7.3 Interpreting the VC dimension

Degrees of freedom. Parameters create degrees of freedom. The number of parameters counts *analog* degrees of freedom; d_{VC} counts the equivalent *binary* degrees of freedom. For positive rays ($h(x) = +1$ to the right of a threshold a), one parameter gives $d_{\text{VC}} = 1$. For positive intervals ($h(x) = +1$ exactly inside an interval), two endpoints give $d_{\text{VC}} = 2$.

Parameters need not contribute degrees of freedom. In a chain of threshold units, each feeding its ± 1 output to the next, only the first unit sees x. The later units receive a binary signal and can only pass it, negate it, or replace it with a constant, so their parameters add no freedom. In this sense d_{VC} measures the *effective* number of parameters.

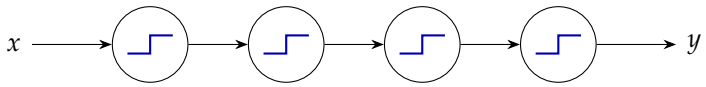


Figure 17: A chain of threshold units. It has several units' worth of parameters but no more freedom than its first unit.

Number of data points needed. The VC inequality contains two small quantities, the tolerance ϵ and the failure probability

$$\delta = 4\, m_{\mathcal{H}}(2N)\, e^{-\frac{1}{8}\epsilon^2 N}.$$

For fixed ϵ and δ, how does the required N depend on d_{VC}? Since $m_{\mathcal{H}}(2N)$ is bounded by a polynomial of degree d_{VC}, the bound behaves like $N^d e^{-N}$ with d playing the role of d_{VC}. Fix $N^d e^{-N}$ at a small value and ask how N changes with d: the crossing points grow roughly in proportion to d. Thus the required N is proportional to d_{VC}, and a rule of thumb is

$$N \geq 10\, d_{\text{VC}}.$$

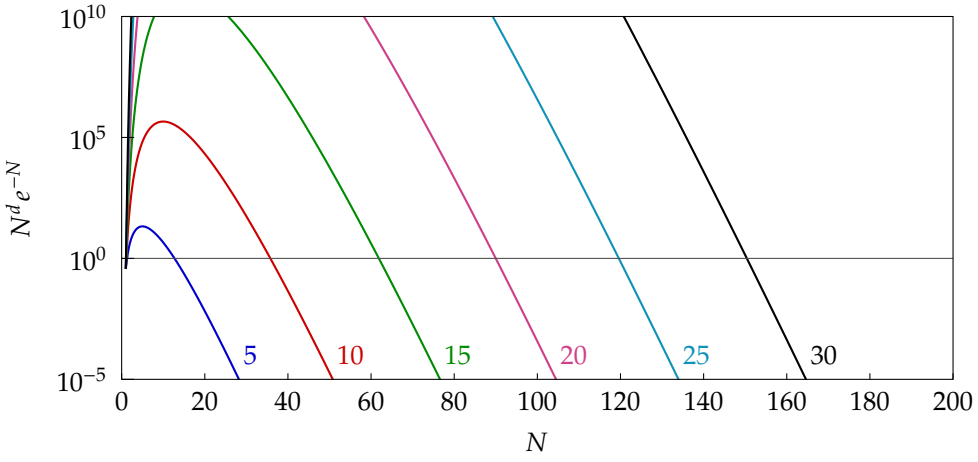


Figure 18: $N^d e^{-N}$ on a logarithmic scale, each curve labeled by its $d \in \{5, 10, \ldots, 30\}$. The horizontal line marks the value 1; the curves cross it at roughly equal spacing in N.

7.4 Generalization bounds

Start from the VC inequality and solve for ϵ in terms of δ:

$$\delta = 4\, m_{\mathcal{H}}(2N)\, e^{-\frac{1}{8}\epsilon^2 N} \quad \Longrightarrow \quad \epsilon = \sqrt{\frac{8}{N} \ln \frac{4\, m_{\mathcal{H}}(2N)}{\delta}} =: \Omega(N, \mathcal{H}, \delta).$$

With probability at least $1 - \delta$, $|E_{\text{out}} - E_{\text{in}}| \leq \Omega(N, \mathcal{H}, \delta)$. Keeping only the useful side gives the generalization bound.

Theorem 7.3 [VC GENERALIZATION BOUND**].** *With probability at least* $1 - \delta$,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \Omega(N, \mathcal{H}, \delta).$$

The penalty Ω grows with the complexity of $\mathcal{H}$, through $m_{\mathcal{H}}(2N)$, and shrinks as N grows.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 7: The VC Dimension, Caltech, April 24, 2012. Lecture slides.

Lecture 8: Bias-Variance Tradeoff

8.1 Approximation versus generalization

A small E_{out} requires a good approximation of f out of sample. The choice of $\mathcal{H}$ pulls in two directions:

- a more complex $\mathcal{H}$ gives a better chance of *approximating* f;
- a less complex $\mathcal{H}$ gives a better chance of *generalizing* out of sample.

The ideal $\mathcal{H} = \{f\}$ would do both, but choosing it amounts to winning the lottery.

VC analysis quantifies the tradeoff as $E_{\text{out}} \leq E_{\text{in}} + \Omega$. Bias-variance analysis is another approach. It decomposes E_{out} into two parts:

1. how well $\mathcal{H}$ can approximate f;
2. how well we can zoom in on a good $h \in \mathcal{H}$ using the data.

It applies to real-valued targets and uses squared error.

8.2 The bias-variance decomposition

Write $g^{(\mathcal{D})}$ for the final hypothesis learned from the data set $\mathcal{D}$. Its out-of-sample error is

$$E_{\text{out}}\big(g^{(\mathcal{D})}\big) = \mathbb{E}_{\mathbf{x}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big].$$

To remove the dependence on a particular data set, take the expectation over $\mathcal{D}$ (data sets of a fixed size N) and exchange the order of the expectations:

$$\mathbb{E}_{\mathcal{D}}\Big[E_{\text{out}}\big(g^{(\mathcal{D})}\big)\Big] = \mathbb{E}_{\mathbf{x}}\Big[\mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big]\Big].$$

It remains to analyze the inner expectation at a fixed $\mathbf{x}$.

Definition 8.1 [Average hypothesis]. The average hypothesis is $\bar{g}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}\big[g^{(\mathcal{D})}(\mathbf{x})\big]$. With many data sets $\mathcal{D}_1, \ldots, \mathcal{D}_K$,

$$\bar{g}(\mathbf{x}) \approx \frac{1}{K}\sum_{k=1}^{K} g^{(\mathcal{D}_k)}(\mathbf{x}).$$

The average hypothesis need not belong to $\mathcal{H}$.

Theorem 8.2 [Bias-variance decomposition]. *At each* $\mathbf{x}$,

$$\mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big] = \underbrace{\mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\big)^2\Big]}_{\text{var}(\mathbf{x})} + \underbrace{\big(\bar{g}(\mathbf{x}) - f(\mathbf{x})\big)^2}_{\text{bias}(\mathbf{x})}.$$

Consequently, $\mathbb{E}_{\mathcal{D}}\big[E_{\text{out}}(g^{(\mathcal{D})})\big] = \mathbb{E}_{\mathbf{x}}[\text{bias}(\mathbf{x}) + \text{var}(\mathbf{x})] = \text{bias} + \text{var}$, *where*

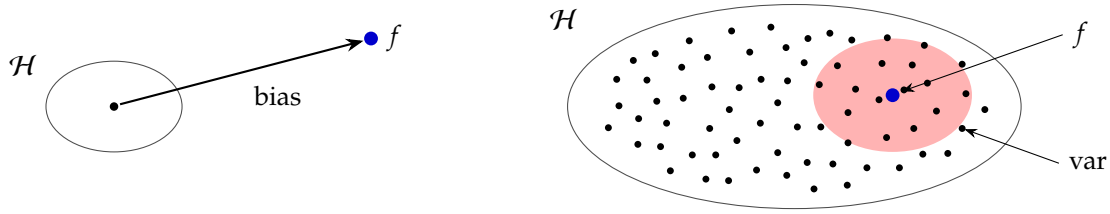
$$\text{bias} = \mathbb{E}_{\mathbf{x}}\Big[\big(\bar{g}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big], \qquad \text{var} = \mathbb{E}_{\mathbf{x}}\Big[\mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\big)^2\Big]\Big].$$

Proof. Add and subtract $\bar{g}(\mathbf{x})$ and expand the square:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big] &= \mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}) + \bar{g}(\mathbf{x}) - f(\mathbf{x})\big)^2\Big] \\ &= \mathbb{E}_{\mathcal{D}}\Big[\big(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\big)^2 + \big(\bar{g}(\mathbf{x}) - f(\mathbf{x})\big)^2 \\ &\qquad + 2\big(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\big)\big(\bar{g}(\mathbf{x}) - f(\mathbf{x})\big)\Big].\end{aligned}$$

The factor $\bar{g}(\mathbf{x}) - f(\mathbf{x})$ does not depend on $\mathcal{D}$, and $\mathbb{E}_{\mathcal{D}}\big[g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\big] = 0$ by the definition of $\bar{g}$, so the cross term vanishes. The middle term is constant in $\mathcal{D}$. Taking $\mathbb{E}_{\mathbf{x}}$ of both sides gives the second statement. □

The bias measures how far the average learned hypothesis is from the target. The variance measures how much the learned hypothesis fluctuates around that average from one data set to another. Enlarging $\mathcal{H}$ moves it closer to f, so the bias goes down; but the learned hypothesis can then land anywhere in a larger set, so the variance goes up.



small $\mathcal{H}$: bias large, variance small — large $\mathcal{H}$: bias small, variance large

Figure 19: Schematic of the tradeoff. A small $\mathcal{H}$ (left) has little spread but sits far from f. A large $\mathcal{H}$ (right) may contain f, but the hypotheses learned from different data sets spread over a wide region around it.

8.3 Example: a sine target

Let $f(x) = \sin(\pi x)$ on $[-1, 1]$, with inputs uniform on $[-1, 1]$, and suppose there are only $N = 2$ training examples. Compare two models:

$$\mathcal{H}_0 : \; h(x) = b, \qquad \mathcal{H}_1 : \; h(x) = ax + b.$$

Approximation. If f were known, the best constant would be $b = 0$, with $E_{\text{out}} = \mathbb{E}[\sin^2(\pi x)] = 0.50$. The best line is $h(x) = (3/\pi)x$, with $E_{\text{out}} = \frac{1}{2} - 3/\pi^2 \approx 0.20$. On approximation alone, $\mathcal{H}_1$ wins.

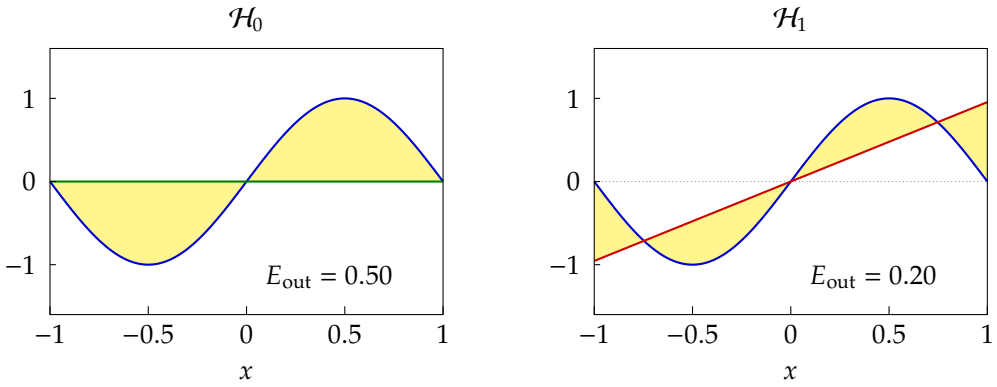
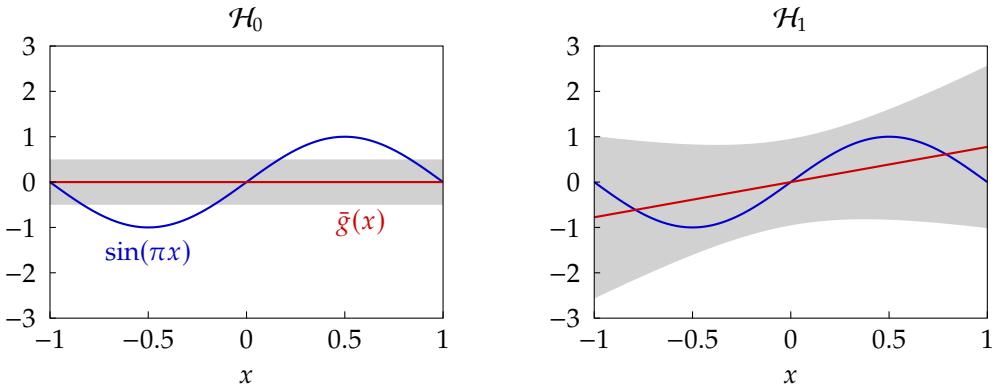


Figure 20: Best approximations of $\sin(\pi x)$ in each model, with the squared-error regions shaded.

Learning. From two examples, $\mathcal{H}_0$ fits the constant $b = \frac{1}{2}(y_1 + y_2)$, and $\mathcal{H}_1$ fits the line through both points. Over random data sets:

- For $\mathcal{H}_0$, $\bar{g}(x) = 0$, so bias = 0.50. The learned constant is the mean of two independent values of $\sin(\pi x)$, each with variance $\frac{1}{2}$, so var = 0.25.
- For $\mathcal{H}_1$, $\bar{g}(x) \approx 0.78x$, with bias ≈ 0.21 and var ≈ 1.68.

The expected out-of-sample errors are 0.75 for $\mathcal{H}_0$ and about 1.9 for $\mathcal{H}_1$. The constant model wins: with two points, the lines fluctuate far more than the constants, and the extra variance outweighs the smaller bias.



bias ≈ 0.50, var ≈ 0.25 bias ≈ 0.21, var ≈ 1.68

Figure 21: Average hypothesis $\bar{g}$ (red) and the band $\bar{g}(x) \pm \sqrt{\text{var}(x)}$ (gray) for $N = 2$, computed by numerical integration over data sets.

Note. Match the model complexity to the data resources, not to the target complexity.

8.4 Learning curves

For a data set $\mathcal{D}$ of size N, the EXPECTED OUT-OF-SAMPLE ERROR $\mathbb{E}_{\mathcal{D}}\left[E_{\text{out}}(g^{(\mathcal{D})})\right]$ and the EXPECTED IN-SAMPLE ERROR $\mathbb{E}_{\mathcal{D}}\left[E_{\text{in}}(g^{(\mathcal{D})})\right]$ are functions of N. Their plots are LEARNING CURVES. As N grows, the expected in-sample error rises and the expected out-of-sample error falls, and the two approach a common level. A simple model reaches that level quickly but the level is higher. A complex model fits small data sets closely, so E_{in} starts near zero and E_{out} starts high, but its curves approach a lower level.

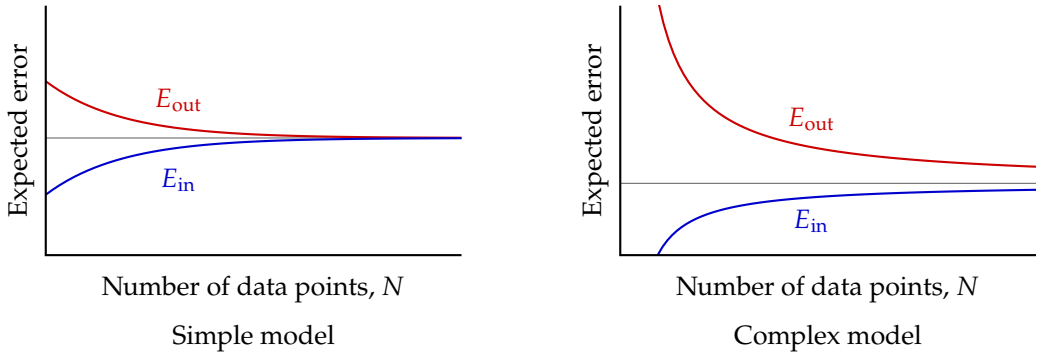


Figure 22: Schematic learning curves for a simple and a complex model. The horizontal line marks the level both curves approach.

The two theories read the same curves differently. VC analysis splits expected E_{out} into the in-sample error and the generalization error, the gap between the curves. Bias-variance analysis splits it into the bias, drawn as a horizontal level, and the variance above it.

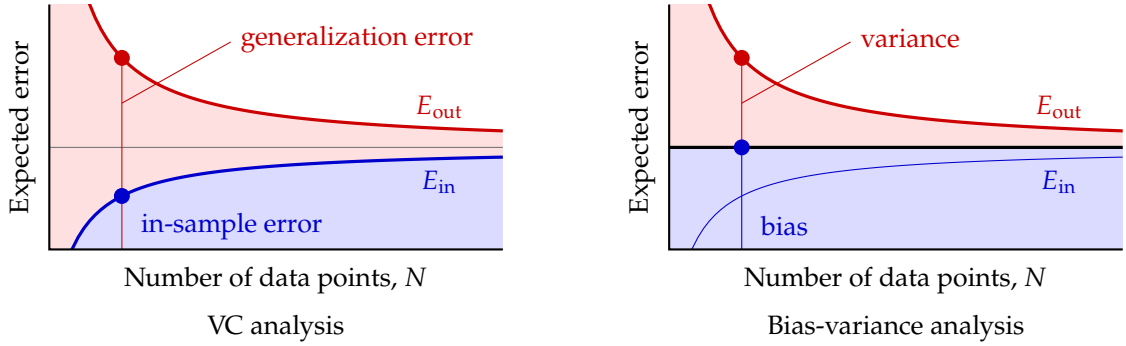


Figure 23: Schematic comparison. Left: VC analysis, where E_{out} is in-sample error plus generalization error. Right: bias-variance analysis, where E_{out} is bias plus variance.

8.5 The linear regression case

Let the target be noisy, $y = \mathbf{w}^{*\mathsf{T}}\mathbf{x} + \text{noise}$, where the noise is independent with mean zero and variance σ^2, and $\mathbf{x} \in \mathbb{R}^{d+1}$ includes the constant coordinate. Given $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, linear regression returns $\mathbf{w} = (\mathrm{X}^\mathsf{T}\mathrm{X})^{-1}\mathrm{X}^\mathsf{T}\mathbf{y}$. Compare two error vectors on the same inputs:

- the in-sample error vector $\mathrm{X}\mathbf{w} - \mathbf{y}$;
- the "out-of-sample" error vector $\mathrm{X}\mathbf{w} - \mathbf{y}'$, where $\mathbf{y}'$ has the same $\mathbf{w}^{*\mathsf{T}}\mathbf{x}_n$ but fresh, independent noise.

Proposition 8.3 [Learning curves for linear regression]. *For $N \geq d+1$, with the mean squared error over the N inputs,*

$$\begin{aligned} \textit{best approximation error} &= \sigma^2, \\ \textit{expected in-sample error} &= \sigma^2\Big(1 - \frac{d+1}{N}\Big), \\ \textit{expected out-of-sample error} &= \sigma^2\Big(1 + \frac{d+1}{N}\Big), \\ \textit{expected generalization error} &= 2\sigma^2\,\frac{d+1}{N}. \end{aligned}$$

Proof. Write $\mathbf{y} = \mathrm{X}\mathbf{w}^* + \boldsymbol{\varepsilon}$ and $\mathbf{y}' = \mathrm{X}\mathbf{w}^* + \boldsymbol{\varepsilon}'$. The matrix $\mathrm{H} = \mathrm{X}(\mathrm{X}^\mathsf{T}\mathrm{X})^{-1}\mathrm{X}^\mathsf{T}$ projects onto the $(d+1)$-dimensional column space of X, and $\mathrm{X}\mathbf{w} = \mathrm{H}\mathbf{y} = \mathrm{X}\mathbf{w}^* + \mathrm{H}\boldsymbol{\varepsilon}$. Even $\mathbf{w}^*$ incurs error σ^2 per point, the best approximation error. The in-sample residual is $-(\mathrm{I} - \mathrm{H})\boldsymbol{\varepsilon}$, and $\mathrm{I} - \mathrm{H}$ is a projection of rank $N - (d+1)$, so its expected squared length is $\sigma^2(N - d - 1)$. The out-of-sample residual is $\mathrm{H}\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}'$; the two parts are independent with expected squared lengths $\sigma^2(d+1)$ and $\sigma^2 N$. Dividing by N gives the in-sample and out-of-sample formulas, and their difference is the generalization error. □

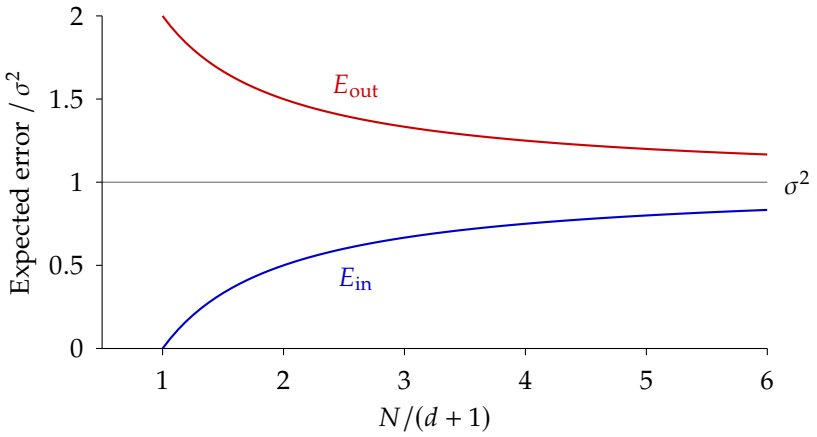


Figure 24: The linear regression learning curves of Proposition 8.3, in units of σ^2, plotted against $N/(d+1)$.

The in-sample error is below the noise level because the fit absorbs part of the noise; the out-of-sample error exceeds it by the same amount. With $N = d + 1$ points the fit is exact, so the expected in-sample error is zero.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 8: Bias-Variance Tradeoff, Caltech, April 26, 2012. Lecture slides.

Lecture 9: The Linear Model II

9.1 Nonlinear transforms and their price

A nonlinear transform maps each input $\mathbf{x} = (x_0, x_1, \dots, x_d)$ to a feature vector

$$\mathbf{z} = \Phi(\mathbf{x}) = (z_0, z_1, \dots, z_{\tilde{d}}), \qquad z_i = \phi_i(\mathbf{x}),$$

for example $\mathbf{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$. A linear model with weights $\tilde{\mathbf{w}}$ is trained on the examples $(\mathbf{z}_n, y_n)$. The final hypothesis, viewed in $\mathcal{X}$, is

$$g(\mathbf{x}) = \operatorname{sign}\big(\tilde{\mathbf{w}}^{\mathsf{T}}\Phi(\mathbf{x})\big) \quad \text{for classification, or} \quad g(\mathbf{x}) = \tilde{\mathbf{w}}^{\mathsf{T}}\Phi(\mathbf{x}) \quad \text{for regression.}$$

The model is still linear in the weights, so the algorithms for linear models apply unchanged in $\mathcal{Z}$.

The price is generalization. Perceptrons in $\mathcal{X}$ have $d_{\text{VC}} = d + 1$, while perceptrons in $\mathcal{Z}$ have $d_{\text{VC}} \le \tilde{d} + 1$. A large $\tilde{d}$ permits elaborate boundaries, but the same guarantee then requires more data.

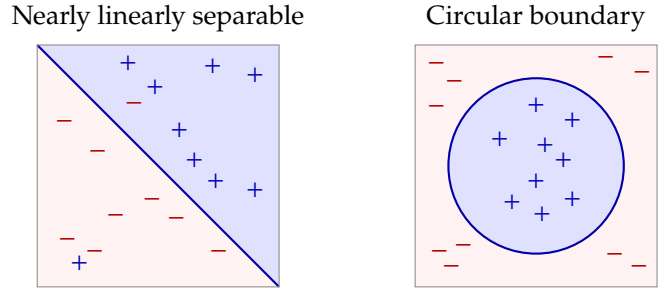


Figure 25: Two schematic non-separable data sets. On the left, a line misclassifies only a few points. On the right, the boundary is genuinely nonlinear.

First case. When a linear boundary fails on only a few points, there are two options: use a linear model in $\mathcal{X}$ and accept $E_{\text{in}} > 0$, or insist on $E_{\text{in}} = 0$ by moving to a high-dimensional $\mathcal{Z}$. The second option fits a few points at the cost of a much larger VC dimension, and its winding boundary is unlikely to generalize.

Second case. A circular boundary calls for a transform. The full quadratic $\mathbf{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$ works. It is tempting to economize after looking at the data: why not $\mathbf{z} = (1, x_1^2, x_2^2)$, or better yet $\mathbf{z} = (1, x_1^2 + x_2^2)$, or even the single feature $z = x_1^2 + x_2^2 - 0.6$? Each choice lowers the nominal VC dimension, but only because the learner has already searched a larger space mentally. The guarantee must account for every hypothesis that was effectively considered, not just the final small set.

Note. DATA SNOOPING: looking at the data *before* choosing the model can be hazardous to E_{out}. A choice of hypothesis set made in response to the data is part of the learning process.

9.2 The logistic regression model

All three linear models compute the SIGNAL $s = \sum_{i=0}^{d} w_i x_i = \mathbf{w}^{\mathsf{T}}\mathbf{x}$ and differ in the output they produce from it. For credit analysis they answer three different questions.

Question	Model	Error measure	Algorithm
Approve or deny	Perceptron	Classification error	PLA, pocket
Amount of credit	Linear regression	Squared error	Pseudo-inverse
Probability of default	Logistic regression	Cross-entropy error	Gradient descent

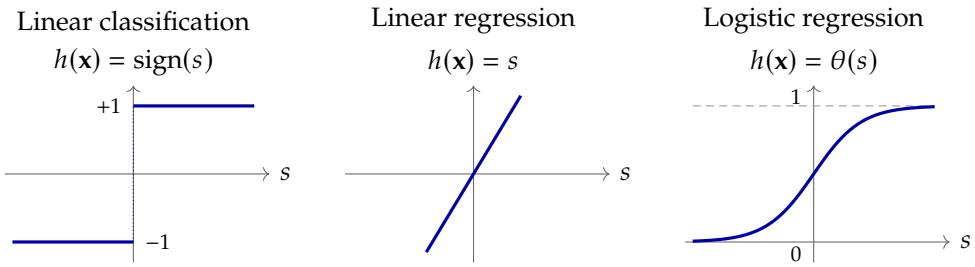


Figure 26: The output of each linear model as a function of the signal s.

Definition 9.1 [LOGISTIC FUNCTION]. The LOGISTIC FUNCTION is

$$\theta(s) = \frac{e^s}{1+e^s} = \frac{1}{1+e^{-s}}.$$

It increases from 0 to 1, with $\theta(0) = \frac{1}{2}$ and $\theta(-s) = 1 - \theta(s)$.

The logistic function is a *soft threshold*: rather than committing to a class, it expresses uncertainty. Its graph is a flattened-out 's', hence the name SIGMOID.

The output $h(\mathbf{x}) = \theta(s)$ is interpreted as a probability. To predict heart attacks, the input $\mathbf{x}$ contains cholesterol level, age, weight, and so on; the signal $s = \mathbf{w}^{\mathsf{T}}\mathbf{x}$ is a risk score, and $\theta(s)$ is the probability of a heart attack.

This probability is genuine. The data $(\mathbf{x}, y)$ have binary $y \in \{-1, +1\}$, generated by a noisy target

$$P(y \mid \mathbf{x}) = \begin{cases} f(\mathbf{x}) & \text{for } y = +1, \\ 1 - f(\mathbf{x}) & \text{for } y = -1. \end{cases}$$

The target $f : \mathbb{R}^d \to [0, 1]$ is itself the probability, and we learn $g(\mathbf{x}) = \theta(\mathbf{w}^\mathsf{T}\mathbf{x}) \approx f(\mathbf{x})$. The examples reveal samples y, never the values $f(\mathbf{x})$.

9.3 Cross-entropy error

A plausible error measure is based on LIKELIHOOD: if $h = f$, how likely is it to observe y from $\mathbf{x}$? The answer is $P(y \mid \mathbf{x}) = h(\mathbf{x})$ for $y = +1$ and $1 - h(\mathbf{x})$ for $y = -1$. Substituting $h(\mathbf{x}) = \theta(\mathbf{w}^\mathsf{T}\mathbf{x})$ and using $\theta(-s) = 1 - \theta(s)$ merges both cases:

$$P(y \mid \mathbf{x}) = \theta(y\, \mathbf{w}^\mathsf{T}\mathbf{x}).$$

For independent examples, the likelihood of $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ is $\prod_{n=1}^{N} \theta(y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n)$. Maximizing it is equivalent to minimizing a decreasing function of it:

$$-\frac{1}{N} \ln \left(\prod_{n=1}^{N} \theta(y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n) \right) = \frac{1}{N} \sum_{n=1}^{N} \ln \frac{1}{\theta(y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n)} = \frac{1}{N} \sum_{n=1}^{N} \ln\bigl(1 + e^{-y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n}\bigr).$$

Definition 9.2 [CROSS-ENTROPY ERROR]. The in-sample CROSS-ENTROPY ERROR of logistic regression is

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} \underbrace{\ln\bigl(1 + e^{-y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n}\bigr)}_{\mathrm{e}(h(\mathbf{x}_n), y_n)}.$$

The pointwise error is small when $y_n \mathbf{w}^\mathsf{T}\mathbf{x}_n$ is large and positive, that is, when the signal agrees confidently with the label, and it grows when they disagree. Equivalently, $\mathrm{e} = \ln \frac{1}{h(\mathbf{x}_n)}$ if $y_n = +1$ and $\ln \frac{1}{1 - h(\mathbf{x}_n)}$ if $y_n = -1$, the cross-entropy between the observed label and the predicted probability.

9.4 Gradient descent

Linear regression minimizes $E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_n (\mathbf{w}^\mathsf{T}\mathbf{x}_n - y_n)^2$ in closed form. The cross-entropy error has no closed-form minimizer and needs an iterative solution.

GRADIENT DESCENT is a general method for nonlinear optimization. Start at $\mathbf{w}(0)$ and take a step along the steepest slope. With a fixed step size η and a unit direction $\hat{\mathbf{v}}$, $\mathbf{w}(1) = \mathbf{w}(0) + \eta\hat{\mathbf{v}}$. To first order,

$$\begin{aligned} \Delta E_{\text{in}} &= E_{\text{in}}\bigl(\mathbf{w}(0) + \eta\hat{\mathbf{v}}\bigr) - E_{\text{in}}\bigl(\mathbf{w}(0)\bigr) \\ &= \eta\, \nabla E_{\text{in}}\bigl(\mathbf{w}(0)\bigr)^\mathsf{T} \hat{\mathbf{v}} + O(\eta^2) \\ &\geq -\eta \left\| \nabla E_{\text{in}}\bigl(\mathbf{w}(0)\bigr) \right\|, \end{aligned}$$

ignoring the $O(\eta^2)$ term in the last line. By Cauchy–Schwarz, since $\hat{\mathbf{v}}$ is a unit vector, equality holds for

$$\hat{\mathbf{v}} = -\frac{\nabla E_{\text{in}}(\mathbf{w}(0))}{\|\nabla E_{\text{in}}(\mathbf{w}(0))\|}.$$

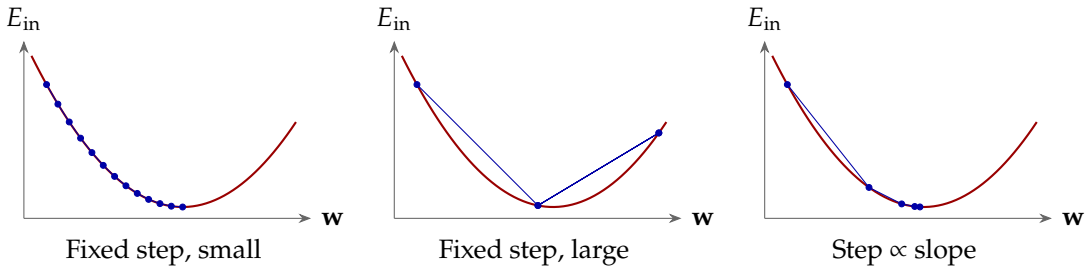


Figure 27: Iterates on a schematic one-dimensional error surface. A small fixed step is slow; a large fixed step bounces across the minimum; steps proportional to the slope are large far away and small near the minimum.

The step size matters. If η is too small, progress is slow; if it is too large, the neglected $O(\eta^2)$ term dominates and the iterates bounce around. The step should therefore increase with the slope. Taking $\eta \propto \|\nabla E_{\text{in}}\|$ cancels the normalization:

$$\Delta\mathbf{w} = -\eta\frac{\nabla E_{\text{in}}(\mathbf{w}(0))}{\|\nabla E_{\text{in}}(\mathbf{w}(0))\|} \quad \text{becomes} \quad \boxed{\Delta\mathbf{w} = -\eta\,\nabla E_{\text{in}}(\mathbf{w}(0))},$$

where η is now a fixed LEARNING RATE. The steps shrink automatically as the gradient vanishes near a minimum.

For the cross-entropy error, with $s_n = y_n\mathbf{w}^{\mathsf{T}}\mathbf{x}_n$,

$$\nabla \ln\bigl(1 + e^{-s_n}\bigr) = \frac{-e^{-s_n}}{1 + e^{-s_n}}\, y_n\mathbf{x}_n = -\frac{y_n\mathbf{x}_n}{1 + e^{s_n}}.$$

Averaging over the examples gives the gradient used below.

Algorithm 9.1: Logistic regression by gradient descent

Input: Examples $(\mathbf{x}_n, y_n)$ with $y_n \in \{-1, +1\}$; learning rate η
Output: Weights $\mathbf{w}$ for $g(\mathbf{x}) = \theta(\mathbf{w}^{\mathsf{T}}\mathbf{x})$

1 Initialize the weights at $t = 0$ to $\mathbf{w}(0)$
2 **for** $t = 0, 1, 2, \ldots$ until it is time to stop **do**
3 $\quad \nabla E_{\text{in}} \leftarrow -\frac{1}{N}\sum_{n=1}^{N} \frac{y_n\mathbf{x}_n}{1 + e^{y_n\mathbf{w}(t)^{\mathsf{T}}\mathbf{x}_n}}$
4 $\quad \mathbf{w}(t+1) \leftarrow \mathbf{w}(t) - \eta\,\nabla E_{\text{in}}$
5 **return** the final weights $\mathbf{w}$

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 9: The Linear Model II, Caltech, May 1, 2012. Lecture slides.

Lecture 10: Neural Networks

10.1 Stochastic gradient descent

Gradient descent minimizes an average of pointwise errors,

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} \mathrm{e}\big(h(\mathbf{x}_n), y_n\big),$$

such as $\mathrm{e} = \ln(1 + e^{-y_n \mathbf{w}^{\mathsf{T}} \mathbf{x}_n})$ in logistic regression, by iterative steps $\Delta\mathbf{w} = -\eta \nabla E_{\text{in}}(\mathbf{w})$. Since ∇E_{in} is based on all examples, this is BATCH GRADIENT DESCENT.

Definition 10.1 [STOCHASTIC GRADIENT DESCENT]. STOCHASTIC GRADIENT DESCENT (SGD) picks one example $(\mathbf{x}_n, y_n)$ at a time, with n chosen uniformly at random, and applies a gradient step to that example's error alone:

$$\Delta\mathbf{w} = -\eta\, \nabla \mathrm{e}\big(h(\mathbf{x}_n), y_n\big).$$

The average direction is the batch direction:

$$\mathbb{E}_n\big[-\nabla \mathrm{e}\big(h(\mathbf{x}_n), y_n\big)\big] = \frac{1}{N}\sum_{n=1}^{N} -\nabla \mathrm{e}\big(h(\mathbf{x}_n), y_n\big) = -\nabla E_{\text{in}}.$$

SGD is thus a randomized version of gradient descent: each step follows $-\nabla E_{\text{in}}$ on average, with random fluctuations around it. Its benefits are:

1. *Cheaper computation*: each step uses one example rather than N.
2. *Randomization*: the fluctuations can carry the weights out of shallow local minima and across flat regions where batch descent would stall.
3. *Simplicity*.

A rule of thumb that works in practice is $\eta = 0.1$.

Example: *Movie ratings.* Represent user i by factors $(u_{i1}, \ldots, u_{iK})$ and movie j by factors $(v_{j1}, \ldots, v_{jK})$, and match them against the observed rating r_{ij} with the error

$$\mathrm{e}_{ij} = \Big(r_{ij} - \sum_{k=1}^{K} u_{ik} v_{jk}\Big)^2.$$

SGD takes one rating at a time. Writing $\varepsilon_{ij} = r_{ij} - \sum_k u_{ik} v_{jk}$ for the residual, the step updates only

the $2K$ factors involved:

$$u_{ik} \leftarrow u_{ik} + 2\eta\, \varepsilon_{ij} v_{jk}, \qquad v_{jk} \leftarrow v_{jk} + 2\eta\, \varepsilon_{ij} u_{ik}, \qquad k = 1, \ldots, K,$$

both computed from the values before the step.

10.2 From perceptrons to multilayer networks

Neural networks borrow the structure of biological neural systems, many simple interconnected units, as a route to their function. The basic unit can be the perceptron of Lecture 1, with outputs in $\{-1, +1\}$.

With inputs $x_1, x_2 \in \{-1, +1\}$, single perceptrons implement the logical operations

$$\mathrm{OR}(x_1, x_2) = \mathrm{sign}(1.5 + x_1 + x_2), \qquad \mathrm{AND}(x_1, x_2) = \mathrm{sign}(-1.5 + x_1 + x_2).$$

OR is -1 only when both inputs are -1; AND is $+1$ only when both are $+1$. Negating an input amounts to flipping the sign of its weight.

Now let h_1 and h_2 be two perceptrons in the plane, and let the target be $+1$ exactly where one of them is $+1$ and the other is -1. Writing products for AND, sums for OR, and bars for negation,

$$f = h_1 \bar{h}_2 + \bar{h}_1 h_2 = \mathrm{OR}\big(\mathrm{AND}(h_1, -h_2),\ \mathrm{AND}(-h_1, h_2)\big).$$

A single perceptron cannot represent f, but layers of perceptrons can. Substituting $h_k(\mathbf{x}) = \mathrm{sign}(\mathbf{w}_k^{\mathsf{T}}\mathbf{x})$ gives a MULTILAYER PERCEPTRON with three layers. It is FEEDFORWARD: signals flow from inputs to output without loops.

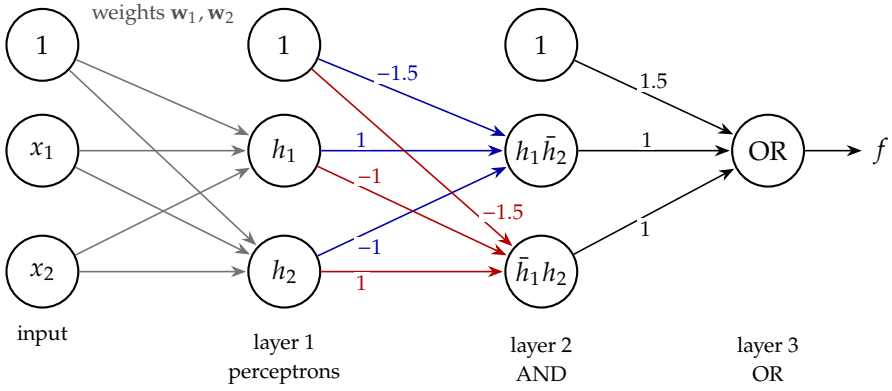


Figure 28: A three-layer perceptron for $f = h_1\bar{h}_2 + \bar{h}_1 h_2$. Every non-constant unit outputs the sign of its weighted input; the constant units supply the thresholds.

Multilayer perceptrons are a powerful model. For a disk-shaped target, the AND of several perceptrons, each a half-plane, carves out a polygon, and more perceptrons approximate the disk

more closely.

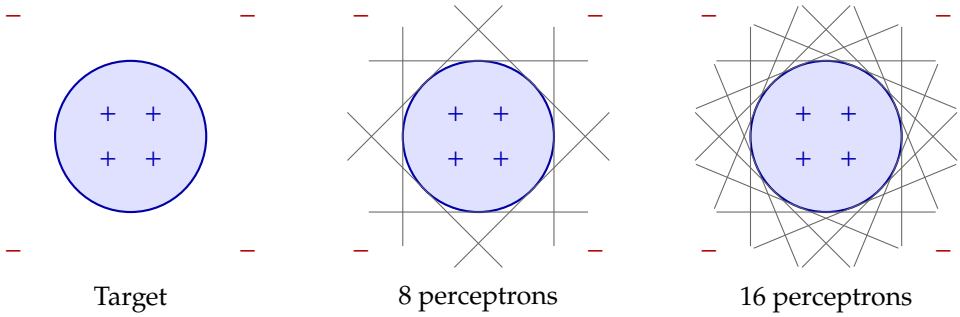


Figure 29: A disk target and polygonal approximations by the intersection of 8 and of 16 half-planes.

This power raises two red flags. For *generalization*, more units mean more weights and a larger VC dimension. For *optimization*, the hard threshold makes the error piecewise constant in the weights, so fitting a network of perceptrons is a combinatorial search. Replacing the hard threshold by a smooth one restores gradient-based training.

10.3 The neural network model

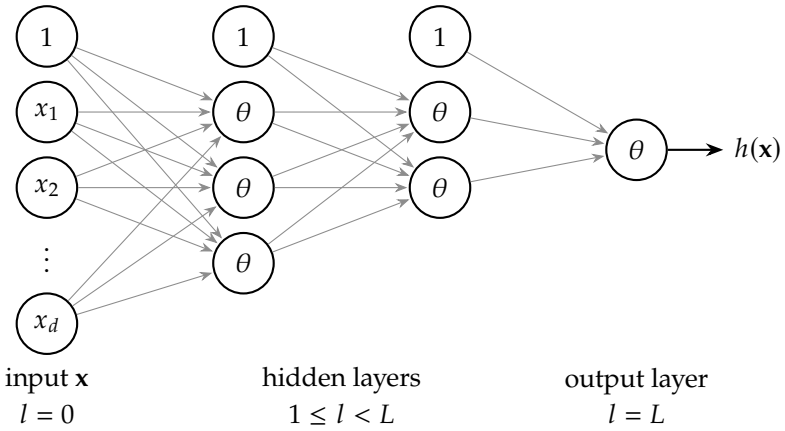


Figure 30: A neural network. Each non-constant unit applies θ to a weighted sum of the previous layer's outputs, including a constant 1.

Definition 10.2 [Neural network]. A neural network has layers $l = 0, 1, \ldots, L$: the input layer $l = 0$, hidden layers $1 \le l < L$, and the output layer $l = L$. Layer l has $d^{(l)}$ units and, below the output layer, a constant unit $x_0^{(l)} = 1$. The weights are

$$w_{ij}^{(l)}, \qquad 1 \le l \le L \text{ (layers)}, \quad 0 \le i \le d^{(l-1)} \text{ (inputs)}, \quad 1 \le j \le d^{(l)} \text{ (outputs)},$$

and each unit computes

$$x_j^{(l)} = \theta\big(s_j^{(l)}\big) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right), \qquad \theta(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}.$$

The input $\mathbf{x}$ is applied to $x_1^{(0)}, \dots, x_{d^{(0)}}^{(0)}$, with $d^{(0)} = d$, and propagated forward layer by layer to $x_1^{(L)} = h(\mathbf{x})$.

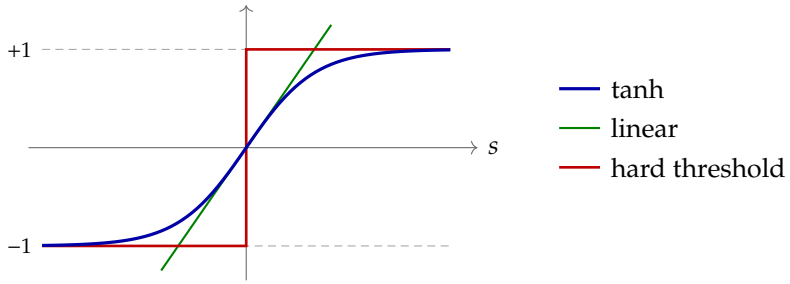


Figure 31: The soft threshold tanh is nearly linear for small signals and close to the hard threshold ±1 for large ones.

10.4 Backpropagation

All the weights $\mathbf{w} = \{w_{ij}^{(l)}\}$ determine $h(\mathbf{x})$, so the error on an example $(\mathbf{x}_n, y_n)$ is a function $\mathrm{e}(h(\mathbf{x}_n), y_n) = \mathrm{e}(\mathbf{w})$. SGD needs its gradient,

$$\nabla \mathrm{e}(\mathbf{w}): \quad \frac{\partial \mathrm{e}(\mathbf{w})}{\partial w_{ij}^{(l)}} \quad \text{for all } i, j, l.$$

Each partial derivative could be evaluated one at a time, analytically or numerically. A trick computes them all efficiently. A weight $w_{ij}^{(l)}$ affects the error only through the signal $s_j^{(l)}$, so

$$\frac{\partial \mathrm{e}(\mathbf{w})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathrm{e}(\mathbf{w})}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)}\, x_i^{(l-1)}, \qquad \delta_j^{(l)} = \frac{\partial \mathrm{e}(\mathbf{w})}{\partial s_j^{(l)}}.$$

The outputs $x_i^{(l-1)}$ come from the forward pass. It remains to compute the δs, starting from the output.

Final layer. For $l = L$ and $j = 1$, take the squared error $\mathrm{e}(\mathbf{w}) = (x_1^{(L)} - y_n)^2$ with $x_1^{(L)} = \theta(s_1^{(L)})$. For tanh, $\theta'(s) = 1 - \theta^2(s)$, so

$$\delta_1^{(L)} = \frac{\partial \mathrm{e}(\mathbf{w})}{\partial s_1^{(L)}} = 2\big(x_1^{(L)} - y_n\big)\big(1 - (x_1^{(L)})^2\big).$$

Proposition 10.3 [Backward recursion]. *For* $1 \le i \le d^{(l-1)}$,

$$\delta_i^{(l-1)} = \left(1 - (x_i^{(l-1)})^2\right) \sum_{j=1}^{d^{(l)}} w_{ij}^{(l)} \delta_j^{(l)}.$$

Proof. The signal $s_i^{(l-1)}$ affects the error only through $x_i^{(l-1)} = \theta(s_i^{(l-1)})$, which feeds every signal $s_j^{(l)}$ of the next layer. By the chain rule,

$$\delta_i^{(l-1)} = \sum_{j=1}^{d^{(l)}} \frac{\partial e(\mathbf{w})}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}} \times \frac{\partial x_i^{(l-1)}}{\partial s_i^{(l-1)}} = \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} \times w_{ij}^{(l)} \times \theta'\left(s_i^{(l-1)}\right),$$

and $\theta'(s_i^{(l-1)}) = 1 - (x_i^{(l-1)})^2$ for tanh. □

The recursion runs the network in reverse: each $\delta_j^{(l)}$ is sent back along the same weights $w_{ij}^{(l)}$ that carried the forward signal. The constant units have no incoming signal, so they need no δ.

Algorithm 10.1: Backpropagation with SGD

Input: Examples $(\mathbf{x}_n, y_n)$; network architecture; learning rate η
Output: Final weights $w_{ij}^{(l)}$

Initialize all weights $w_{ij}^{(l)}$ at random
for $t = 0, 1, 2, \ldots$ until it is time to stop **do**
  Pick $n \in \{1, 2, \ldots, N\}$
  Forward: compute all $x_j^{(l)}$ from $\mathbf{x}_n$
  Backward: compute all $\delta_j^{(l)}$, from $l = L$ down to $l = 1$
  Update the weights: $w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta\, x_i^{(l-1)} \delta_j^{(l)}$
return the final weights $w_{ij}^{(l)}$

Each pass uses every weight once, so one forward and one backward pass yield the entire gradient. Random initialization matters: if all weights start at zero, every hidden output $x_j^{(l)}$ with $j \ge 1$ is $\tanh(0) = 0$ and every hidden δ is zero, so no update ever moves the weights except the output unit's threshold.

Hidden layers as a learned transform. The hidden layers compute a nonlinear transform of the input, and the output unit is a linear model on top of it. Unlike the transform Φ of Lecture 9, this transform is learned from the data. Its hidden units, however, generally lack a clear interpretation.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 10: Neural Networks, Caltech, May 3, 2012. Lecture slides.

Lecture 11: Overfitting

11.1 What is overfitting?

Take a simple target, a quadratic, and five data points with a little noise. A fourth-order polynomial passes through all five points, so $E_{\text{in}} = 0$, yet it swings far from the target between and beyond them, so E_{out} is huge.

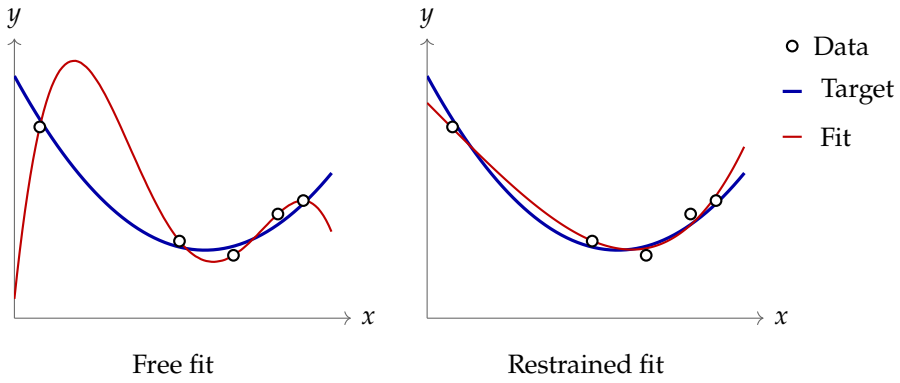


Figure 32: Schematic data from a quadratic target with small noise. Left: the fourth-order polynomial through all five points. Right: a fourth-order fit restrained from fitting the data exactly.

Overfitting is a comparative notion. When a neural network is trained on noisy data, E_{in} keeps decreasing with the training epochs, while E_{out} first decreases and then increases. Overfitting is the regime where

$$E_{\text{in}} \downarrow \qquad E_{\text{out}} \uparrow .$$

Bad generalization, a large gap between E_{out} and E_{in}, describes a single hypothesis. Overfitting describes moving from one hypothesis to another that fits the data better but performs worse out of sample. Stopping at the minimum of E_{out} is EARLY STOPPING.

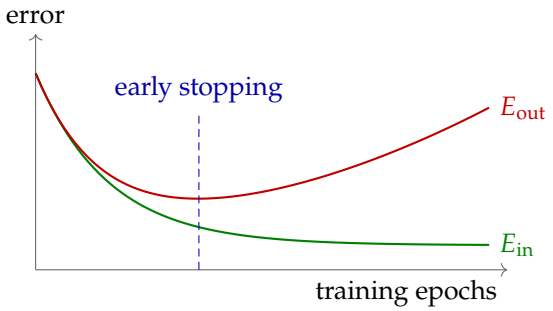


Figure 33: Schematic training and out-of-sample errors of a neural network on noisy data. Beyond the dashed line, training reduces E_{in} but increases E_{out}.

Definition: Overfitting is fitting the data more than is warranted. The culprit is fitting the noise, which is harmful: the fit spends its freedom on patterns that do not carry over to new inputs.

11.2 The role of noise

Consider two targets, each sampled at fifteen points: a 10th-order polynomial with noise, and a noiseless 50th-order polynomial. Fit each data set with $\mathcal{H}_2$ and $\mathcal{H}_{10}$, the polynomials of order 2 and 10.

	Noisy 10th-order target		Noiseless 50th-order target	
	$\mathcal{H}_2$	$\mathcal{H}_{10}$	$\mathcal{H}_2$	$\mathcal{H}_{10}$
E_{in}	0.050	0.034	0.029	10^{-5}
E_{out}	0.127	9.00	0.120	7680

An irony of two learners. Learners O and R both know that the first target is 10th order. O chooses $\mathcal{H}_{10}$ to match it; R chooses the restricted $\mathcal{H}_2$. Although O's hypothesis set contains the target's form, R wins by a wide margin. Learning curves explain why: with few data points, the larger model's E_{out} is far worse, even though it would eventually be better.

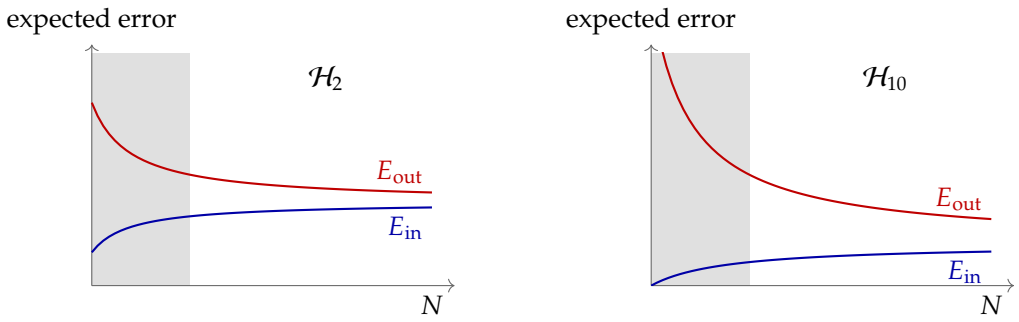


Figure 34: Schematic learning curves. In the shaded range of small N, $\mathcal{H}_{10}$ has the larger E_{out}.

Even without noise. For the noiseless 50th-order target, the same learners know there is no noise, and $\mathcal{H}_2$ still wins. Is there really no noise? The part of the target that neither model can capture plays the same role, as the next subsection makes precise.

A detailed experiment. Generate data from

$$y = f(\mathbf{x}) + \epsilon(\mathbf{x}) = \sum_{q=0}^{Q_f} \alpha_q x^q + \epsilon(\mathbf{x}),$$

with f normalized and ϵ of variance σ^2. Three quantities vary: the noise level σ^2, the target complexity Q_f, and the data set size N. Fitting each data set with $\mathcal{H}_2$ and $\mathcal{H}_{10}$ gives g_2 and g_{10}, and

the OVERFIT MEASURE is

$$E_{\text{out}}(g_{10}) - E_{\text{out}}(g_2).$$

Over a grid of (N, σ^2) and of (N, Q_f), the measure is largest for small N with a large noise level or a complex target, and it falls as N grows.

Increase		Overfitting
Number of data points N	$\uparrow$	$\downarrow$
Stochastic noise σ^2	$\uparrow$	$\uparrow$
Deterministic noise (via Q_f)	$\uparrow$	$\uparrow$

11.3 Deterministic noise

Definition 11.1 [DETERMINISTIC NOISE]. Let $h^* \in \mathcal{H}$ be the best approximation to f in $\mathcal{H}$. The DETERMINISTIC NOISE is the part of f that $\mathcal{H}$ cannot capture:

$$f(\mathbf{x}) - h^*(\mathbf{x}).$$

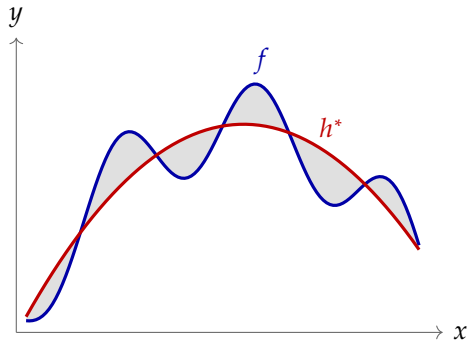


Figure 35: A schematic target f and its best quadratic approximation h^* in mean squared error. The shaded gap is the deterministic noise for $\mathcal{H}_2$.

It is noise because, with finite N, the learner cannot tell it apart from stochastic noise: $\mathcal{H}$ tries to fit it and is harmed in the same way. It differs from stochastic noise in two respects:

1. it depends on $\mathcal{H}$: a richer hypothesis set captures more of f;
2. it is fixed for a given $\mathbf{x}$: drawing the same input again gives the same value, whereas $\epsilon(\mathbf{x})$ is redrawn.

For a fixed $\mathcal{H}$, deterministic noise grows with Q_f, which explains the overfitting seen for complex noiseless targets.

Noise and bias–variance. Recall the decomposition at a point $\mathbf{x}$,

$$\mathbb{E}_{\mathcal{D}}\left[(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x}))^2\right] = \underbrace{\mathbb{E}_{\mathcal{D}}\left[(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2\right]}_{\text{var}(\mathbf{x})} + \underbrace{\left(\bar{g}(\mathbf{x}) - f(\mathbf{x})\right)^2}_{\text{bias}(\mathbf{x})},$$

where $\bar{g}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[g^{(\mathcal{D})}(\mathbf{x})]$. Now let the target be noisy: $y = f(\mathbf{x}) + \epsilon(\mathbf{x})$ with $\mathbb{E}[\epsilon(\mathbf{x})] = 0$.

Proposition 11.2 [Two noise terms]. *If the noise $\epsilon(\mathbf{x})$ at the test point is independent of the training set $\mathcal{D}$, then*

$$\mathbb{E}_{\mathcal{D},\epsilon,\mathbf{x}}\left[(g^{(\mathcal{D})}(\mathbf{x}) - y)^2\right] = \underbrace{\mathbb{E}_{\mathcal{D},\mathbf{x}}\left[(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2\right]}_{\text{var}} + \underbrace{\mathbb{E}_{\mathbf{x}}\left[(\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2\right]}_{\text{bias}} + \underbrace{\mathbb{E}_{\epsilon,\mathbf{x}}\left[\epsilon(\mathbf{x})^2\right]}_{\sigma^2}.$$

Proof. Fix $\mathbf{x}$ and write

$$g^{(\mathcal{D})}(\mathbf{x}) - y = \left(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\right) + \left(\bar{g}(\mathbf{x}) - f(\mathbf{x})\right) - \epsilon(\mathbf{x}).$$

Squaring gives the three squares plus cross terms. Each cross term has expectation zero: $\mathbb{E}_{\mathcal{D}}[g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})] = 0$ by the definition of $\bar{g}$, and $\mathbb{E}[\epsilon(\mathbf{x})] = 0$ with $\epsilon(\mathbf{x})$ independent of $\mathcal{D}$. Taking the expectation over $\mathbf{x}$ completes the proof. □

The bias term is the deterministic noise, with $\bar{g}$ in the role of h^*, and σ^2 is the stochastic noise. Neither can be removed by the learner. The variance term is where finite N hurts: by fitting either kind of noise in the sample, $g^{(\mathcal{D})}$ varies more from one data set to another.

11.4 Dealing with overfitting

There are two cures.

- Regularization puts on the brakes: it restrains the fit so that it cannot follow the data too closely. In Figure 32, the restrained fit stays close to the target.
- Validation checks the bottom line: it estimates E_{out} directly and uses that estimate to choose.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 11: Overfitting, Caltech, May 8, 2012. Lecture slides.

Lecture 12: Regularization

Overfitting fits the data more than is warranted. Regularization counters it by restraining the fit. It has two views: mathematically, it comes from ill-posed problems in function approximation; heuristically, it *handicaps* the minimization of E_{in} so that the learning algorithm cannot chase every fluctuation in the data.

12.1 A familiar example

Return to the target $\sin(\pi x)$ on $[-1, 1]$, learned from $N = 2$ examples with lines $h(x) = ax + b$. Fitting each data set exactly produces lines that vary wildly from one data set to the next. Restraining the fit toward smaller weights makes the fitted lines flatter and much less variable. In bias and variance terms:

	bias	var
without regularization	0.21	1.69
with regularization	0.23	0.33

Regularization costs a little bias and saves a lot of variance, so the expected out-of-sample error drops from 1.90 to 0.56.

12.2 The polynomial model and weight constraints

For $x \in [-1, 1]$, let $\mathcal{H}_Q$ be the polynomials of order Q, written in the Legendre basis:

$$\mathbf{z} = \big(1, L_1(x), \dots, L_Q(x)\big)^{\mathsf{T}}, \qquad \mathcal{H}_Q = \left\{ \sum_{q=0}^{Q} w_q L_q(x) \right\},$$

with $L_1(x) = x$, $L_2(x) = \frac{1}{2}(3x^2 - 1)$, $L_3(x) = \frac{1}{2}(5x^3 - 3x)$, $L_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3)$ and $L_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x)$. Learning is linear regression in the $\mathcal{Z}$ space. Transform (x_n, y_n) to $(\mathbf{z}_n, y_n)$ and let Z be the matrix with rows $\mathbf{z}_n^{\mathsf{T}}$. Without a constraint we minimize

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{w}^{\mathsf{T}} \mathbf{z}_n - y_n)^2 = \frac{1}{N} (Z\mathbf{w} - \mathbf{y})^{\mathsf{T}} (Z\mathbf{w} - \mathbf{y}), \qquad \mathbf{w}_{\text{lin}} = (Z^{\mathsf{T}} Z)^{-1} Z^{\mathsf{T}} \mathbf{y}.$$

A hard constraint sets weights to zero: $\mathcal{H}_2$ is the version of $\mathcal{H}_{10}$ with $w_q = 0$ for $q > 2$. The softer

SOFT-ORDER CONSTRAINT limits the total size of the weights:

$$\text{minimize } \frac{1}{N}(Z\mathbf{w}-\mathbf{y})^{\mathsf{T}}(Z\mathbf{w}-\mathbf{y}) \quad \text{subject to} \quad \mathbf{w}^{\mathsf{T}}\mathbf{w} = \sum_{q=0}^{Q} w_q^2 \le C.$$

Its solution is denoted $\mathbf{w}_{\text{reg}}$, in place of $\mathbf{w}_{\text{lin}}$.

12.3 From constrained to augmented error

If $\mathbf{w}_{\text{lin}}^{\mathsf{T}}\mathbf{w}_{\text{lin}} \le C$, the constraint does nothing and $\mathbf{w}_{\text{reg}} = \mathbf{w}_{\text{lin}}$. Otherwise $\mathbf{w}_{\text{reg}}$ lies on the sphere $\mathbf{w}^{\mathsf{T}}\mathbf{w} = C$, where the level sets of the quadratic E_{in} are ellipsoids centered at $\mathbf{w}_{\text{lin}}$.

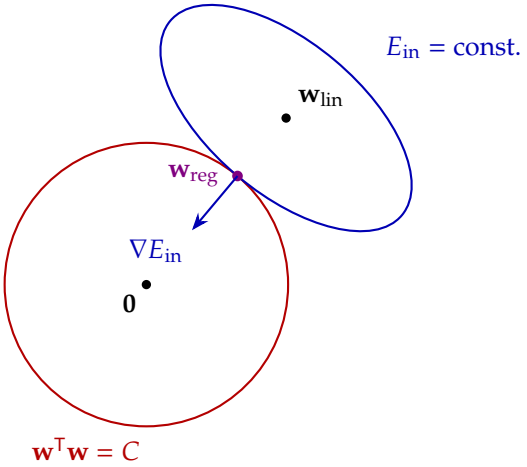


Figure 36: Schematic constrained minimum. At $\mathbf{w}_{\text{reg}}$ the contour of E_{in} touches the constraint circle, so ∇E_{in} is normal to the circle and points inward, opposite to $\mathbf{w}_{\text{reg}}$.

At the constrained minimum, ∇E_{in} has no component tangent to the sphere; otherwise moving along the sphere would lower E_{in}. It also cannot point outward, since stepping inward along $-\nabla E_{\text{in}}$ would stay feasible and lower E_{in}. Hence $\nabla E_{\text{in}}(\mathbf{w}_{\text{reg}}) \propto -\mathbf{w}_{\text{reg}}$. Writing the constant as $2\lambda/N$ with $\lambda \ge 0$,

$$\nabla E_{\text{in}}(\mathbf{w}_{\text{reg}}) + \frac{2\lambda}{N}\mathbf{w}_{\text{reg}} = \mathbf{0}.$$

This is the condition for a minimum of the AUGMENTED ERROR

$$E_{\text{aug}}(\mathbf{w}) = E_{\text{in}}(\mathbf{w}) + \frac{\lambda}{N}\mathbf{w}^{\mathsf{T}}\mathbf{w},$$

which is convex, and strictly convex for $\lambda > 0$. Minimizing E_{aug} *unconditionally* therefore solves the constrained problem, with λ determined by C. A larger budget C corresponds to a smaller λ; $\lambda = 0$ when the constraint is inactive. The constrained form defines a restricted hypothesis set, which is what the VC analysis addresses; the augmented form is what we compute.

Proposition 12.1 [Regularized linear regression]. *For $\lambda > 0$, the unique minimizer of $E_{\text{aug}}(\mathbf{w}) = \frac{1}{N}\big((Z\mathbf{w} - \mathbf{y})^\mathsf{T}(Z\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^\mathsf{T}\mathbf{w}\big)$ is*

$$\boxed{\mathbf{w}_{\text{reg}} = (Z^\mathsf{T}Z + \lambda \mathrm{I})^{-1} Z^\mathsf{T}\mathbf{y},}$$

as opposed to $\mathbf{w}_{\text{lin}} = (Z^\mathsf{T}Z)^{-1}Z^\mathsf{T}\mathbf{y}$ without regularization.

Proof. $\nabla E_{\text{aug}}(\mathbf{w}) = \frac{2}{N}\big(Z^\mathsf{T}(Z\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}\big) = \mathbf{0}$ gives $(Z^\mathsf{T}Z + \lambda \mathrm{I})\mathbf{w} = Z^\mathsf{T}\mathbf{y}$. The matrix is positive definite, hence invertible, since $\mathbf{v}^\mathsf{T}(Z^\mathsf{T}Z + \lambda \mathrm{I})\mathbf{v} = \|Z\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0$ for $\mathbf{v} \neq \mathbf{0}$. □

Fitting the same noisy data with a high-order polynomial for $\lambda = 0,\ 0.0001,\ 0.01,\ 1$ moves from overfitting to underfitting. At $\lambda = 0$ the fit swings far from the target between data points; a tiny $\lambda = 0.0001$ already removes the swings and nearly matches the target; by $\lambda = 1$ the fit is too flat to follow the data. The amount of regularization matters as much as its form.

12.4 Weight decay

Minimizing $E_{\text{in}}(\mathbf{w}) + \frac{\lambda}{N}\mathbf{w}^\mathsf{T}\mathbf{w}$ is called weight decay. A gradient descent step on E_{aug} is

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla E_{\text{in}}\big(\mathbf{w}(t)\big) - 2\eta\frac{\lambda}{N}\mathbf{w}(t) = \mathbf{w}(t)\Big(1 - 2\eta\frac{\lambda}{N}\Big) - \eta\nabla E_{\text{in}}\big(\mathbf{w}(t)\big)\ :$$

before the usual step, the weights shrink, or decay, by a constant factor. The same applies in neural networks, with $\mathbf{w}^\mathsf{T}\mathbf{w} = \sum_{l=1}^{L}\sum_{i=0}^{d^{(l-1)}}\sum_{j=1}^{d^{(l)}}\big(w_{ij}^{(l)}\big)^2$ summing over all weights.

Variations weight some coefficients more than others:

$$\Omega(\mathbf{w}) = \sum_{q=0}^{Q}\gamma_q w_q^2.$$

With $\gamma_q = 2^q$ high-order coefficients are expensive, favoring a low-order fit; $\gamma_q = 2^{-q}$ favors a high-order fit. In neural networks, different layers can get different γ's. The general quadratic form is the Tikhonov regularizer $\mathbf{w}^\mathsf{T}\Gamma^\mathsf{T}\Gamma\mathbf{w}$.

Not every constraint helps. *Weight growth*, which rewards large weights, makes the expected E_{out} worse as soon as $\lambda > 0$, while weight decay first lowers it and only later raises it. The practical rule: stochastic noise is "high-frequency" and deterministic noise is also non-smooth, so constrain learning toward *smoother* hypotheses.

12.5 The general augmented error

For a regularizer $\Omega = \Omega(h)$, we minimize

$$E_{\text{aug}}(h) = E_{\text{in}}(h) + \frac{\lambda}{N}\Omega(h).$$

This mirrors the VC form $E_{\text{out}}(h) \leq E_{\text{in}}(h) + \Omega(\mathcal{H})$, with a penalty on the individual hypothesis in place of a penalty on the whole set. E_{aug} is therefore a better proxy for E_{out} than E_{in} is.

12.6 Choosing a regularizer

The perfect regularizer would constrain learning in the direction of the target function, which is unknown; requiring it would be circular. The guiding principle is to constrain toward *smoother* or *simpler* hypotheses, as noise is not smooth. If Ω is a poor choice, λ still limits the damage: a λ chosen by validation can drive the penalty toward zero. Regularization is a necessary evil: it gives up some fit to the data to avoid fitting noise.

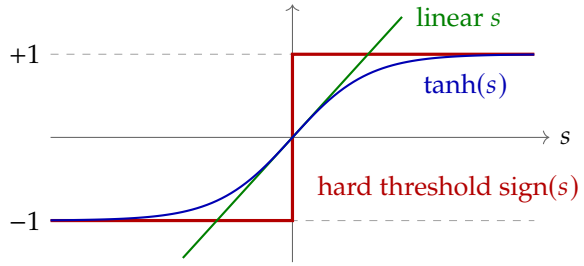


Figure 37: The neuron output $\tanh(s)$ lies between its linear approximation near $s = 0$ and the hard threshold. Small weights keep signals s in the linear regime; large weights make the neuron logical.

Neural-network regularizers. Weight decay moves a network from linear toward logical: small weights keep tanh near its linear part, so the network computes a simple, nearly linear function, while larger weights allow hard-threshold behavior. Weight elimination sets weights to zero; fewer weights mean a smaller VC dimension. The soft version is

$$\Omega(\mathbf{w}) = \sum_{i,j,l} \frac{\left(w_{ij}^{(l)}\right)^2}{\beta^2 + \left(w_{ij}^{(l)}\right)^2}.$$

A weight much smaller than β costs about $(w_{ij}^{(l)}/\beta)^2$, like weight decay, and is pushed toward zero; a weight much larger than β costs about 1, regardless of its size, and is left alone.

Early stopping. Stopping gradient descent before E_{in} is minimized restricts how far the weights travel, so it regularizes through the optimizer. When to stop is decided by validation.

The optimal λ. In experiments that vary the noise level, the λ minimizing the expected E_{out} increases with the noise. With stochastic noise $\sigma^2 = 0,\ 0.25,\ 0.5$, the best λ is 0 without noise and moves right as σ^2 grows. Deterministic noise behaves the same way: as the target complexity grows through $Q_f = 15,\ 30,\ 100$, the best λ grows from 0. In both cases the noise is what regularization protects against, and more noise calls for more regularization.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 12: Regularization, Caltech, May 10, 2012. Lecture slides.

Lecture 13: Validation

In one form or another,

$$E_{\text{out}}(h) = E_{\text{in}}(h) + \text{overfit penalty}.$$

Regularization estimates the overfit penalty and adds a proxy for it to E_{in}. Validation instead estimates $E_{\text{out}}(h)$ directly.

13.1 The validation set

On an out-of-sample point $(\mathbf{x}, y)$, the error of h is $\mathrm{e}\big(h(\mathbf{x}), y\big)$, for example the squared error $\big(h(\mathbf{x}) - y\big)^2$ or the binary error $\mathbf{1}\{h(\mathbf{x}) \neq y\}$. For a random point,

$$\mathbb{E}\big[\mathrm{e}(h(\mathbf{x}), y)\big] = E_{\text{out}}(h), \qquad \mathrm{Var}\big[\mathrm{e}(h(\mathbf{x}), y)\big] = \sigma^2.$$

On a validation set $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_K, y_K)$, drawn independently of h, the validation error is

$$E_{\text{val}}(h) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{e}\big(h(\mathbf{x}_k), y_k\big).$$

Since the K errors are independent with the same mean and variance,

$$\mathbb{E}\big[E_{\text{val}}(h)\big] = E_{\text{out}}(h), \qquad \mathrm{Var}\big[E_{\text{val}}(h)\big] = \frac{\sigma^2}{K}, \qquad E_{\text{val}}(h) = E_{\text{out}}(h) \pm O\left(\frac{1}{\sqrt{K}}\right).$$

For binary error, $\sigma^2 = E_{\text{out}}(h)\big(1 - E_{\text{out}}(h)\big) \leq \frac{1}{4}$.

K is taken out of N. Given $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, reserve K points as $\mathcal{D}_{\text{val}}$ and train on the remaining $N - K$ points, $\mathcal{D}_{\text{train}}$. A small K gives a bad estimate. A large K leaves fewer training points, so by the learning curve the hypothesis being estimated has a higher expected E_{out}.

K is put back into N. Write g^- for the hypothesis learned from $\mathcal{D}_{\text{train}}$ and g for the one learned from all of $\mathcal{D}$. We report g, and use

$$E_{\text{val}} = E_{\text{val}}(g^-)$$

as its estimate. This estimate concerns g^-, not g: with a large K, g^- is trained on far fewer points than g, so $E_{\text{val}}(g^-)$ becomes a bad estimate of $E_{\text{out}}(g)$. A rule of thumb is $K = N/5$.

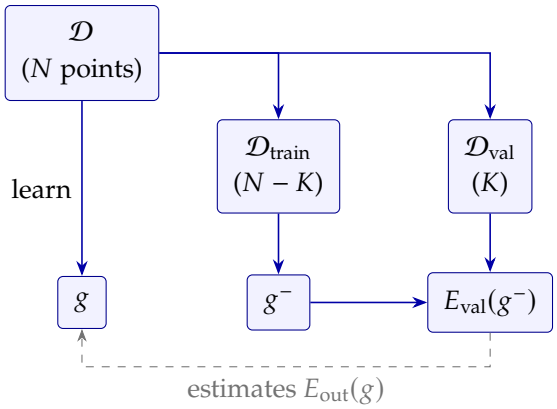


Figure 38: $\mathcal{D}_{\text{train}}$ produces g^-, which is evaluated on $\mathcal{D}_{\text{val}}$. The final hypothesis g is learned from all N points, and $E_{\text{val}}(g^-)$ estimates $E_{\text{out}}(g)$.

13.2 Why "validation"

$\mathcal{D}_{\text{val}}$ is used to make learning choices, such as when to stop training in early stopping. Once an estimate of E_{out} affects learning, the set is no longer a *test* set; it becomes a *validation* set. A test set gives an unbiased estimate; a validation set has an optimistic bias.

Example: Let h_1 and h_2 have $E_{\text{out}}(h_1) = E_{\text{out}}(h_2) = 0.5$, and let their error estimates $\mathrm{e}_1, \mathrm{e}_2$ be independent and uniform on $[0, 1]$. Pick the hypothesis with the smaller estimate, reporting $\mathrm{e} = \min(\mathrm{e}_1, \mathrm{e}_2)$. Since $\mathbb{P}(\mathrm{e} > t) = (1-t)^2$,

$$\mathbb{E}[\mathrm{e}] = \int_0^1 (1-t)^2 \, dt = \frac{1}{3} < 0.5,$$

although either chosen hypothesis has $E_{\text{out}} = 0.5$.

13.3 Model selection

Validation can be used more than once to choose among models $\mathcal{H}_1, \ldots, \mathcal{H}_M$, which may differ in their hypothesis sets or in parameters such as λ.

Algorithm 13.1: Model selection with a validation set

Input: Data $\mathcal{D}$ of size N, models $\mathcal{H}_1, \ldots, \mathcal{H}_M$, validation size K
Output: Final hypothesis g_{m^*}

Split $\mathcal{D}$ into $\mathcal{D}_{\text{train}}$ ($N-K$ points) and $\mathcal{D}_{\text{val}}$ (K points)
for $m = 1, \ldots, M$ **do**
  $g_m^- \leftarrow$ hypothesis learned from $\mathcal{D}_{\text{train}}$ using $\mathcal{H}_m$
  $E_m \leftarrow E_{\text{val}}(g_m^-)$
$m^* \leftarrow \arg\min_m E_m$
return g_{m^*}, learned from all of $\mathcal{D}$ using $\mathcal{H}_{m^*}$

The bias. Because $\mathcal{H}_{m^*}$ was selected using $\mathcal{D}_{\text{val}}$, $E_{\text{val}}(g_{m^*}^-)$ is an optimistically biased estimate of $E_{\text{out}}(g_{m^*}^-)$. When selecting between two models with a fixed N, the expected $E_{\text{val}}(g_{m^*}^-)$ lies below the expected $E_{\text{out}}(g_{m^*}^-)$. Both rise with K, since the finalists train on fewer points, and the gap narrows as the larger validation set makes each estimate more reliable.

How much bias. Selection uses $\mathcal{D}_{\text{val}}$ for "training" on the FINALISTS MODEL $\mathcal{H}_{\text{val}} = \{g_1^-, g_2^-, \dots, g_M^-\}$. This set is fixed before $\mathcal{D}_{\text{val}}$ is examined, so the finite-hypothesis bound of Lecture 2 applies with K examples and M hypotheses: with probability at least $1 - \delta$,

$$E_{\text{out}}(g_{m^*}^-) \leq E_{\text{val}}(g_{m^*}^-) + \sqrt{\frac{\ln(2M/\delta)}{2K}} = E_{\text{val}}(g_{m^*}^-) + O\left(\sqrt{\frac{\ln M}{K}}\right).$$

When validation chooses a continuous parameter, such as the regularization parameter λ or the early-stopping time T, the finalists form an infinite set with one degree of freedom, and the VC analysis applies in place of $\ln M$.

Data contamination. E_{in}, E_{test}, and E_{val} all estimate E_{out}. CONTAMINATION is the optimistic, deceptive bias that results when the data behind an estimate also influenced the learning. The training set is totally contaminated, the validation set slightly contaminated, and the test set totally clean.

13.4 Cross validation

The chain

$$E_{\text{out}}(g) \underset{\text{small } K}{\approx} E_{\text{out}}(g^-) \underset{\text{large } K}{\approx} E_{\text{val}}(g^-)$$

shows the dilemma: the first approximation needs K small, the second K large. Cross validation achieves both effects by reusing the data.

Definition 13.1 [LEAVE-ONE-OUT CROSS VALIDATION]. For $n = 1, \dots, N$, let $\mathcal{D}_n = \mathcal{D} \setminus \{(\mathbf{x}_n, y_n)\}$ and let g_n^- be the hypothesis learned from $\mathcal{D}_n$. Its error on the left-out point is $\mathrm{e}_n = E_{\text{val}}(g_n^-) = \mathrm{e}\big(g_n^-(\mathbf{x}_n), y_n\big)$. The CROSS VALIDATION ERROR is

$$E_{\text{cv}} = \frac{1}{N} \sum_{n=1}^{N} \mathrm{e}_n.$$

Each g_n^- trains on $N - 1$ points, so it is close to g, while E_{cv} averages N validation errors. These errors are not independent, but E_{cv} uses all N points for evaluation.

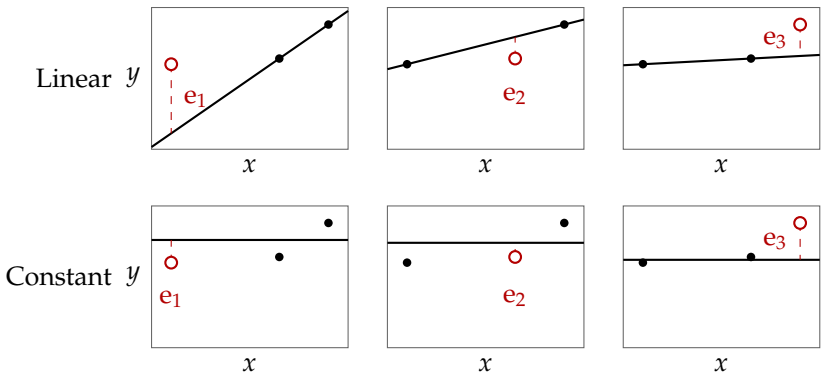


Figure 39: Leave-one-out on three schematic points. Each panel fits the model to the two filled points and measures the error e_n (dashed) on the open point. For these points the constant model has the smaller $E_{\text{cv}} = \frac{1}{3}(e_1 + e_2 + e_3)$ under squared error, so cross validation selects it.

Model selection using cross validation. Compute E_{cv} for each model and select the smallest, as with E_{val} above. In the figure, the linear model extrapolates badly when an end point is left out, and the constant model wins.

Example: *Digits.* Classify handwritten digits as 1 or not 1 using average intensity and symmetry, $(1, x_1, x_2)$. The fifth-order transform

$$(1, x_1, x_2) \to (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, \ldots, x_1^5, x_1^4x_2, x_1^3x_2^2, x_1^2x_2^3, x_1x_2^4, x_2^5)$$

gives 20 features beyond the constant. Model k uses the first k of them. As k increases, E_{in} decreases to zero, while E_{out} and E_{cv} rise beyond a small number of features; E_{cv} follows the shape of E_{out}. Using all features gives $E_{\text{in}} = 0\%$ and $E_{\text{out}} = 2.5\%$ with a contorted boundary. Choosing the number of features by cross validation gives $E_{\text{in}} = 0.8\%$ and $E_{\text{out}} = 1.5\%$ with a smooth one.

Leave more than one out. Leave-one-out needs N training sessions on $N - 1$ points each. Instead, split $\mathcal{D}$ into N/K folds of K points; validate on each fold in turn after training on the rest, and average the N/K validation errors. This takes N/K training sessions on $N - K$ points each. A common choice is 10-FOLD CROSS VALIDATION, $K = N/10$.

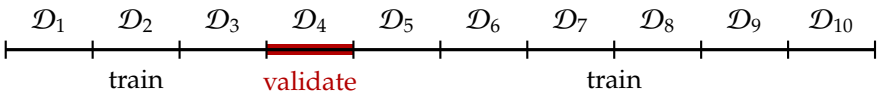


Figure 40: One round of 10-fold cross validation: fold $\mathcal{D}_4$ validates and the other nine folds train. Each fold validates once.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 13: Validation, Caltech, May 15, 2012. Lecture slides.

Lecture 14: Support Vector Machines

14.1 Maximizing the margin

Linearly separable data admit many separating lines. They all have zero training error, but they differ in their MARGIN: the distance from the line to the nearest data point, or half the width of the widest separating strip around the line. Which line is best? This raises two questions: why a bigger margin is better, and which $\mathbf{w}$ maximizes it.

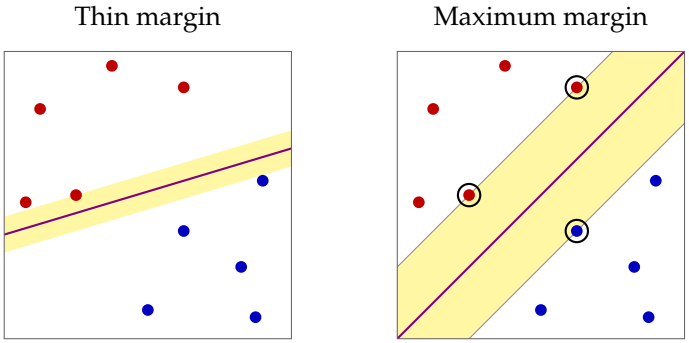


Figure 41: Two separating lines for the same schematic data. The thin margin on the left barely clears the points; the maximum-margin line on the right is as far as possible from both classes. Circled points are its support vectors.

Why a bigger margin is better. Recall the growth function. Lines can produce all 8 dichotomies of three points in general position. If a dichotomy must instead be produced by a line with a *fat* margin, a strip of a required width containing no data point, then dichotomies whose classes are too close together become impossible. Fat margins imply fewer dichotomies, hence a smaller effective growth function and better generalization.

Two technicalities. Let $\mathbf{x}_n$ be the data point nearest to the plane $\mathbf{w}^\mathsf{T}\mathbf{x} = 0$. How far is it?

1. *Normalize* $\mathbf{w}$. Scaling $\mathbf{w}$ does not change the plane, so require $|\mathbf{w}^\mathsf{T}\mathbf{x}_n| = 1$ for the nearest point.
2. *Pull out* w_0. From here on, $\mathbf{w} = (w_1, \ldots, w_d)$ and the bias b is kept apart, with no coordinate x_0. The plane is now $\mathbf{w}^\mathsf{T}\mathbf{x} + b = 0$.

Proposition 14.1 [DISTANCE TO THE PLANE]. *If* $|\mathbf{w}^\mathsf{T}\mathbf{x}_n + b| = 1$, *then the distance between* $\mathbf{x}_n$ *and the plane* $\mathbf{w}^\mathsf{T}\mathbf{x} + b = 0$ *is* $1/\|\mathbf{w}\|$.

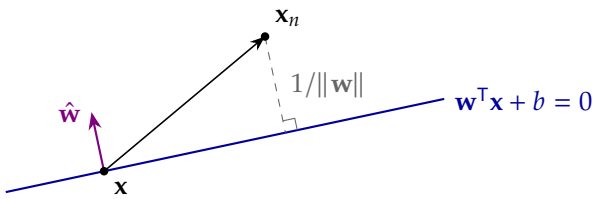


Figure 42: The normal $\hat{\mathbf{w}}$ and the projection of $\mathbf{x}_n - \mathbf{x}$ onto it, for any point $\mathbf{x}$ on the plane.

Proof. For $\mathbf{x}'$ and $\mathbf{x}''$ on the plane, $\mathbf{w}^\mathsf{T}\mathbf{x}' + b = 0$ and $\mathbf{w}^\mathsf{T}\mathbf{x}'' + b = 0$, so $\mathbf{w}^\mathsf{T}(\mathbf{x}' - \mathbf{x}'') = 0$: $\mathbf{w}$ is orthogonal to every direction in the plane. The distance is therefore the length of the projection of $\mathbf{x}_n - \mathbf{x}$ onto $\hat{\mathbf{w}} = \mathbf{w}/\|\mathbf{w}\|$, for any $\mathbf{x}$ on the plane:

$$\text{distance} = \left|\hat{\mathbf{w}}^\mathsf{T}(\mathbf{x}_n - \mathbf{x})\right| = \frac{1}{\|\mathbf{w}\|}\left|\mathbf{w}^\mathsf{T}\mathbf{x}_n - \mathbf{w}^\mathsf{T}\mathbf{x}\right| = \frac{1}{\|\mathbf{w}\|}\left|\mathbf{w}^\mathsf{T}\mathbf{x}_n + b - \mathbf{w}^\mathsf{T}\mathbf{x} - b\right| = \frac{1}{\|\mathbf{w}\|}.$$

□

The optimization problem. We want to

$$\text{maximize } \frac{1}{\|\mathbf{w}\|} \quad \text{subject to} \quad \min_{n=1,\dots,N} |\mathbf{w}^\mathsf{T}\mathbf{x}_n + b| = 1.$$

For a plane that separates the data correctly, $|\mathbf{w}^\mathsf{T}\mathbf{x}_n + b| = y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b)$. The problem becomes

$$\boxed{\text{minimize } \tfrac{1}{2}\mathbf{w}^\mathsf{T}\mathbf{w} \quad \text{subject to} \quad y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) \geq 1 \ \text{ for } n = 1, \dots, N,}$$

over $\mathbf{w} \in \mathbb{R}^d$, $b \in \mathbb{R}$. Relaxing "min = 1" to "≥ 1" changes nothing: if every constraint were slack at a solution, dividing $(\mathbf{w}, b)$ by $\min_n y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) > 1$ would stay feasible and reduce $\mathbf{w}^\mathsf{T}\mathbf{w}$.

14.2 The solution

This is constrained optimization with inequality constraints, so instead of plain Lagrange multipliers we use the KKT conditions. The situation resembles regularization (Lecture 12) with the roles exchanged: regularization optimizes E_{in} under a constraint on $\mathbf{w}^\mathsf{T}\mathbf{w}$, while the SVM optimizes $\mathbf{w}^\mathsf{T}\mathbf{w}$ under a constraint on E_{in}.

Lagrange formulation. Minimize

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\mathsf{T}\mathbf{w} - \sum_{n=1}^{N} \alpha_n \big(y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) - 1\big)$$

with respect to $\mathbf{w}$ and b, and maximize with respect to each $\alpha_n \geq 0$. Setting the derivatives in $\mathbf{w}$ and b to zero,

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n = \mathbf{0}, \qquad \frac{\partial \mathcal{L}}{\partial b} = -\sum_{n=1}^{N} \alpha_n y_n = 0.$$

Substitute $\mathbf{w} = \sum_n \alpha_n y_n \mathbf{x}_n$ and $\sum_n \alpha_n y_n = 0$ in $\mathcal{L}$. The b term vanishes, and $\sum_n \alpha_n y_n \mathbf{w}^\mathsf{T} \mathbf{x}_n = \mathbf{w}^\mathsf{T} \mathbf{w}$, so $\mathcal{L} = \sum_n \alpha_n - \frac{1}{2} \mathbf{w}^\mathsf{T} \mathbf{w}$:

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \mathbf{x}_n^\mathsf{T} \mathbf{x}_m.$$

Maximize $\mathcal{L}(\boldsymbol{\alpha})$ with respect to $\boldsymbol{\alpha}$ subject to $\alpha_n \geq 0$ for $n = 1, \ldots, N$ and $\sum_{n=1}^{N} \alpha_n y_n = 0$.

Quadratic programming. Negating the objective gives a standard quadratic program:

$$\min_{\boldsymbol{\alpha}} \ \frac{1}{2} \boldsymbol{\alpha}^\mathsf{T} \begin{bmatrix} y_1 y_1 \, \mathbf{x}_1^\mathsf{T} \mathbf{x}_1 & y_1 y_2 \, \mathbf{x}_1^\mathsf{T} \mathbf{x}_2 & \cdots & y_1 y_N \, \mathbf{x}_1^\mathsf{T} \mathbf{x}_N \\ y_2 y_1 \, \mathbf{x}_2^\mathsf{T} \mathbf{x}_1 & y_2 y_2 \, \mathbf{x}_2^\mathsf{T} \mathbf{x}_2 & \cdots & y_2 y_N \, \mathbf{x}_2^\mathsf{T} \mathbf{x}_N \\ \vdots & \vdots & \ddots & \vdots \\ y_N y_1 \, \mathbf{x}_N^\mathsf{T} \mathbf{x}_1 & y_N y_2 \, \mathbf{x}_N^\mathsf{T} \mathbf{x}_2 & \cdots & y_N y_N \, \mathbf{x}_N^\mathsf{T} \mathbf{x}_N \end{bmatrix} \boldsymbol{\alpha} + (-\mathbf{1}^\mathsf{T}) \boldsymbol{\alpha}$$

subject to the linear constraint $\mathbf{y}^\mathsf{T} \boldsymbol{\alpha} = 0$ and the bounds $\mathbf{0} \leq \boldsymbol{\alpha} \leq \infty$. The matrix holds the quadratic coefficients and $-\mathbf{1}^\mathsf{T}$ the linear ones.

QP hands us $\boldsymbol{\alpha}$. From the solution $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_N)$, $\mathbf{w} = \sum_{n=1}^{N} \alpha_n y_n \mathbf{x}_n$. The KKT condition

$$\alpha_n \big(y_n (\mathbf{w}^\mathsf{T} \mathbf{x}_n + b) - 1 \big) = 0, \qquad n = 1, \ldots, N,$$

says that each point either has $\alpha_n = 0$ or lies exactly on the margin. This is the same dichotomy as in regularization, where $\lambda = 0$ unless the constraint is active. A point with $\alpha_n > 0$ is a SUPPORT VECTOR.

Support vectors. The support vectors are the closest points to the plane: they achieve the margin, $y_n(\mathbf{w}^\mathsf{T} \mathbf{x}_n + b) = 1$. Only they contribute to the weights,

$$\mathbf{w} = \sum_{\mathbf{x}_n \text{ is SV}} \alpha_n y_n \mathbf{x}_n,$$

and b follows from any support vector: since $y_n = \pm 1$, $y_n(\mathbf{w}^\mathsf{T} \mathbf{x}_n + b) = 1$ gives $b = y_n - \mathbf{w}^\mathsf{T} \mathbf{x}_n$.

Algorithm 14.1: Maximum-margin separation by quadratic programming

Input: Linearly separable data $(\mathbf{x}_n, y_n)$, $n = 1, \dots, N$, with $y_n \in \{-1, +1\}$
Output: The maximum-margin classifier

$Q_{nm} \leftarrow y_n y_m \mathbf{x}_n^\mathsf{T} \mathbf{x}_m$ for $n, m = 1, \dots, N$
$\boldsymbol{\alpha} \leftarrow \arg\min_{\boldsymbol{\alpha}} \frac{1}{2}\boldsymbol{\alpha}^\mathsf{T} Q \boldsymbol{\alpha} - \mathbf{1}^\mathsf{T}\boldsymbol{\alpha}$ subject to $\mathbf{y}^\mathsf{T}\boldsymbol{\alpha} = 0$, $\boldsymbol{\alpha} \geq \mathbf{0}$
$\mathbf{w} \leftarrow \sum_{n:\,\alpha_n > 0} \alpha_n y_n \mathbf{x}_n$
choose any s with $\alpha_s > 0$
$b \leftarrow y_s - \mathbf{w}^\mathsf{T}\mathbf{x}_s$
return $g(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$

14.3 Nonlinear transforms

With a transform $\mathbf{z} = \Phi(\mathbf{x})$, use $\mathbf{z}$ instead of $\mathbf{x}$:

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \mathbf{z}_n^\mathsf{T} \mathbf{z}_m .$$

The data enter only through inner products $\mathbf{z}_n^\mathsf{T}\mathbf{z}_m$, and the quadratic program still has N variables, however large the dimension of $\mathcal{Z}$. Data that are not linearly separable in $\mathcal{X}$ can become separable in $\mathcal{Z}$, and the SVM finds the maximum-margin plane there.

Support vectors in $\mathcal{X}$ space. Support vectors live in $\mathcal{Z}$ space. In $\mathcal{X}$ space we see their "pre-images", and the boundary can be a complicated curve. The margin is maintained in $\mathcal{Z}$ space, not in $\mathcal{X}$ space.

Theorem 14.2 [Generalization through support vectors]. *For the maximum-margin classifier,*

$$\mathbb{E}[E_{\text{out}}] \leq \frac{\mathbb{E}[\#\text{ of SV's}]}{N-1},$$

where E_{out} is that of the hypothesis learned from $N-1$ points and the support vectors are counted for N points.

Proof. Use leave-one-out cross validation (Lecture 13) with binary error. If $\mathbf{x}_n$ is not a support vector, then $\alpha_n = 0$; the full solution remains feasible and satisfies the KKT conditions without $\mathbf{x}_n$, so $g_n^- = g$, which classifies $\mathbf{x}_n$ correctly, and $\mathrm{e}_n = 0$. Hence $E_{\text{cv}} \leq$ # of SV's$/N$. Each e_n is the error of a hypothesis learned from $N-1$ points on a fresh point, so $\mathbb{E}[E_{\text{cv}}] = \mathbb{E}[E_{\text{out}}]$ for training size $N-1$. Therefore $\mathbb{E}[E_{\text{out}}] \leq \mathbb{E}[\#\text{ of SV's}]/N \leq \mathbb{E}[\#\text{ of SV's}]/(N-1)$. □

The bound does not involve the dimension of $\mathcal{Z}$. A transform to a very high-dimensional space can still generalize well if the solution has few support vectors.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 14: Support Vector Machines, Caltech, May 17, 2012. Lecture slides.

Lecture 15: Kernel Methods

A nonlinear transform $\mathbf{z} = \Phi(\mathbf{x})$ lets the support vector machine fit a complex boundary while its generalization is controlled by the support vectors. This lecture removes the cost of computing in $\mathcal{Z}$ and relaxes the requirement that the data be separable.

15.1 What the SVM needs from $\mathcal{Z}$

In the transformed space, the dual problem maximizes

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \, \mathbf{z}_n^{\mathsf{T}} \mathbf{z}_m$$

subject to $\alpha_n \geq 0$ for all n and $\sum_n \alpha_n y_n = 0$. The final hypothesis is

$$g(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^{\mathsf{T}} \mathbf{z} + b), \qquad \mathbf{w} = \sum_{\mathbf{z}_n \text{ is SV}} \alpha_n y_n \mathbf{z}_n,$$

with b determined by $y_m(\mathbf{w}^{\mathsf{T}} \mathbf{z}_m + b) = 1$ for a support vector $\mathbf{z}_m$. Evaluating g needs $\mathbf{z}_n^{\mathsf{T}} \mathbf{z}$, and solving for b and $\boldsymbol{\alpha}$ needs $\mathbf{z}_n^{\mathsf{T}} \mathbf{z}_m$. The vectors $\mathbf{z}$ themselves never appear except inside inner products.

Definition 15.1 [Kernel]. For $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$ with $\mathbf{z} = \Phi(\mathbf{x})$ and $\mathbf{z}' = \Phi(\mathbf{x}')$, the kernel is the generalized inner product

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{z}^{\mathsf{T}} \mathbf{z}'.$$

Example: For $\mathbf{x} = (x_1, x_2)$ and the second-order transform $\Phi(\mathbf{x}) = (1, x_1, x_2, x_1^2, x_2^2, x_1 x_2)$,

$$K(\mathbf{x}, \mathbf{x}') = 1 + x_1 x_1' + x_2 x_2' + x_1^2 x_1'^2 + x_2^2 x_2'^2 + x_1 x_1' x_2 x_2'.$$

15.2 The kernel trick

The question is whether $K(\mathbf{x}, \mathbf{x}')$ can be computed *without* transforming $\mathbf{x}$ and $\mathbf{x}'$. Consider

$$\begin{aligned} K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^{\mathsf{T}} \mathbf{x}')^2 &= (1 + x_1 x_1' + x_2 x_2')^2 \\ &= 1 + x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2 x_1 x_1' + 2 x_2 x_2' + 2 x_1 x_1' x_2 x_2'. \end{aligned}$$

This is the inner product of $\Phi(\mathbf{x}) = (1, x_1^2, x_2^2, \sqrt{2}\, x_1, \sqrt{2}\, x_2, \sqrt{2}\, x_1 x_2)$ with the same transform of $\mathbf{x}'$: the same monomials as before, with different scaling. It costs one inner product in $\mathcal{X}$ and a squaring.

More generally, for $\mathcal{X} = \mathbb{R}^d$ and a polynomial transform of order Q, the POLYNOMIAL KERNEL is

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\mathsf{T}\mathbf{x}')^Q = (1 + x_1 x_1' + \cdots + x_d x_d')^Q.$$

Expanding the power gives a positively weighted sum of products of monomials of degree at most Q, so it is an inner product of scaled monomial vectors. For $d = 10$ and $Q = 100$, there are $\binom{110}{10} \approx 4.69 \times 10^{13}$ such monomials, yet the kernel needs only about d multiplications and one power. The scale can be adjusted as $K(\mathbf{x}, \mathbf{x}') = (a\mathbf{x}^\mathsf{T}\mathbf{x}' + b)^Q$.

15.3 An infinite-dimensional $\mathcal{Z}$

If $K(\mathbf{x}, \mathbf{x}')$ is an inner product in *some* space $\mathcal{Z}$, the SVM machinery applies; $\mathcal{Z}$ only needs to exist. The GAUSSIAN KERNEL (also called the RBF kernel) is

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right), \qquad \gamma > 0.$$

For scalar x and $\gamma = 1$,

$$\exp\left(-(x - x')^2\right) = \exp(-x^2)\exp(-x'^2) \underbrace{\sum_{k=0}^{\infty} \frac{2^k x^k x'^k}{k!}}_{\exp(2xx')},$$

which is the inner product of the infinite sequences with coordinates $\Phi_k(x) = \exp(-x^2)\sqrt{2^k/k!}\, x^k$, $k = 0, 1, 2, \ldots$

Applying this kernel to slightly non-separable data transforms $\mathcal{X}$ into an infinite-dimensional $\mathcal{Z}$. This need not be overkill: by the support-vector bound of Lecture 14,

$$\mathbb{E}[E_{\text{out}}] \le \frac{\mathbb{E}[\#\text{ of SVs}]}{N - 1},$$

which does not involve the dimension of $\mathcal{Z}$. Counting the support vectors in the solution gives an in-sample check of generalization.

15.4 Kernel formulation of the SVM

The quadratic program is unchanged except for its quadratic coefficients $y_n y_m K(\mathbf{x}_n, \mathbf{x}_m)$, which form the matrix

$$\begin{bmatrix} y_1 y_1 K(\mathbf{x}_1, \mathbf{x}_1) & \cdots & y_1 y_N K(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ y_N y_1 K(\mathbf{x}_N, \mathbf{x}_1) & \cdots & y_N y_N K(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}.$$

Substituting $\mathbf{w} = \sum_{\alpha_n > 0} \alpha_n y_n \mathbf{z}_n$ into $g(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\mathsf{T}\mathbf{z} + b)$ expresses the final hypothesis through the kernel alone:

$$g(\mathbf{x}) = \operatorname{sign}\left(\sum_{\alpha_n > 0} \alpha_n y_n K(\mathbf{x}_n, \mathbf{x}) + b \right), \qquad b = y_m - \sum_{\alpha_n > 0} \alpha_n y_n K(\mathbf{x}_n, \mathbf{x}_m)$$

for any support vector ($\alpha_m > 0$). The formula for b follows from $y_m(\mathbf{w}^\mathsf{T}\mathbf{z}_m + b) = 1$ because $1/y_m = y_m$ for $y_m \in \{-1, +1\}$.

Algorithm 15.1: SVM with a kernel

Input: Examples $(\mathbf{x}_n, y_n)$ with $y_n \in \{-1, +1\}$; a valid kernel K
Output: Final hypothesis g

$Q_{nm} \leftarrow y_n y_m K(\mathbf{x}_n, \mathbf{x}_m)$ for $n, m = 1, \dots, N$
$\boldsymbol{\alpha} \leftarrow$ maximizer of $\mathbf{1}^\mathsf{T}\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\mathsf{T} Q \boldsymbol{\alpha}$ subject to $\boldsymbol{\alpha} \geq \mathbf{0}$, $\mathbf{y}^\mathsf{T}\boldsymbol{\alpha} = 0$
choose m with $\alpha_m > 0$
$b \leftarrow y_m - \sum_{\alpha_n > 0} \alpha_n y_n K(\mathbf{x}_n, \mathbf{x}_m)$
return $g(\mathbf{x}) = \operatorname{sign}\left(\sum_{\alpha_n > 0} \alpha_n y_n K(\mathbf{x}_n, \mathbf{x}) + b\right)$

15.5 Valid kernels

A function $K(\mathbf{x}, \mathbf{x}')$ is a VALID KERNEL if it is an inner product $\Phi(\mathbf{x})^\mathsf{T}\Phi(\mathbf{x}')$ for some $\mathcal{Z}$. Validity can be shown by construction, as for the polynomial and Gaussian kernels above, or by a mathematical criterion.

Theorem 15.2 [MERCER'S CONDITION]. *K is a valid kernel if and only if it is symmetric, $K(\mathbf{x}, \mathbf{x}') = K(\mathbf{x}', \mathbf{x})$, and for every choice of points $\mathbf{x}_1, \dots, \mathbf{x}_N$ the matrix with entries $K(\mathbf{x}_n, \mathbf{x}_m)$ is positive semi-definite.*

Proof. Necessity: if $K(\mathbf{x}, \mathbf{x}') = \mathbf{z}^\mathsf{T}\mathbf{z}'$, then K is symmetric, and for any coefficients $c_1, \dots, c_N$,

$$\sum_{n=1}^{N} \sum_{m=1}^{N} c_n c_m K(\mathbf{x}_n, \mathbf{x}_m) = \left\| \sum_{n=1}^{N} c_n \mathbf{z}_n \right\|^2 \geq 0.$$

Sufficiency, the construction of $\mathcal{Z}$ from such a K, is the content of Mercer's theorem and is taken as given here. □

15.6 Soft-margin SVM

Data can be non-separable in two ways. *Slightly* non-separable data have a few points on the wrong side of a reasonable linear boundary; *seriously* non-separable data need a nonlinear boundary. The second case calls for a transform or kernel; the first calls for tolerating a few errors. In practice both can occur together.

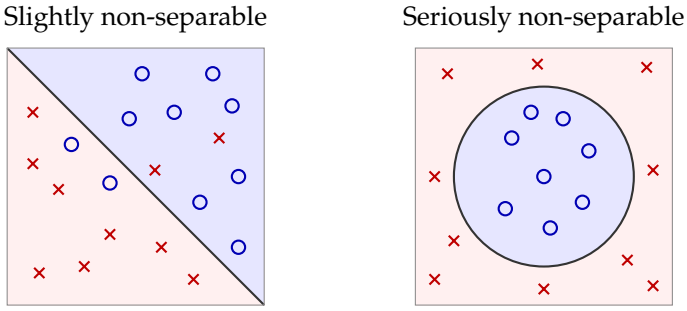


Figure 43: Schematic data of the two kinds. Left: a linear boundary with a few errors. Right: a boundary that must be nonlinear.

A MARGIN VIOLATION occurs when $y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) \geq 1$ fails. Quantify it with a SLACK $\xi_n \geq 0$:

$$y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) \geq 1 - \xi_n, \qquad \text{total violation} = \sum_{n=1}^{N} \xi_n.$$

The soft-margin SVM solves

$$\begin{aligned} &\text{minimize} \quad \frac{1}{2}\mathbf{w}^\mathsf{T}\mathbf{w} + C\sum_{n=1}^{N}\xi_n \\ &\text{subject to} \quad y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) \geq 1 - \xi_n \quad \text{and} \quad \xi_n \geq 0, \qquad n = 1, \dots, N, \end{aligned}$$

over $\mathbf{w} \in \mathbb{R}^d$, $b \in \mathbb{R}$, $\boldsymbol{\xi} \in \mathbb{R}^N$. The constant $C > 0$ sets the price of violations relative to a wide margin.

The Lagrangian, minimized over $\mathbf{w}, b, \boldsymbol{\xi}$ and maximized over $\alpha_n \geq 0$ and $\beta_n \geq 0$, is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2}\mathbf{w}^\mathsf{T}\mathbf{w} + C\sum_{n=1}^{N}\xi_n - \sum_{n=1}^{N}\alpha_n\big(y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) - 1 + \xi_n\big) - \sum_{n=1}^{N}\beta_n\xi_n.$$

Setting derivatives to zero gives

$$\nabla_{\mathbf{w}}\mathcal{L} = \mathbf{w} - \sum_{n=1}^{N}\alpha_n y_n \mathbf{x}_n = \mathbf{0}, \qquad \frac{\partial\mathcal{L}}{\partial b} = -\sum_{n=1}^{N}\alpha_n y_n = 0, \qquad \frac{\partial\mathcal{L}}{\partial \xi_n} = C - \alpha_n - \beta_n = 0.$$

The last condition makes the coefficient $C - \alpha_n - \beta_n$ of every ξ_n vanish, so the slacks drop out of $\mathcal{L}$, and $\beta_n = C - \alpha_n \geq 0$ becomes the constraint $\alpha_n \leq C$. What remains is the hard-margin dual with a box constraint.

Theorem 15.3 [SOFT-MARGIN DUAL]. *Maximize*

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{n=1}^{N}\alpha_n - \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} y_n y_m \alpha_n \alpha_m \, \mathbf{x}_n^\mathsf{T}\mathbf{x}_m$$

subject to $0 \le \alpha_n \le C$ *for* $n = 1, \ldots, N$ *and* $\sum_n \alpha_n y_n = 0$. *Then* $\mathbf{w} = \sum_n \alpha_n y_n \mathbf{x}_n$ *minimizes* $\frac{1}{2}\mathbf{w}^\mathsf{T}\mathbf{w} + C \sum_n \xi_n$.

With a kernel, replace $\mathbf{x}_n^\mathsf{T}\mathbf{x}_m$ by $K(\mathbf{x}_n, \mathbf{x}_m)$ exactly as before.

Complementary slackness, $\alpha_n\big(y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) - 1 + \xi_n\big) = 0$ and $\beta_n \xi_n = 0$, sorts the support vectors ($\alpha_n > 0$) into two types.

- Margin support vectors, $0 < \alpha_n < C$: then $\beta_n > 0$ forces $\xi_n = 0$, so $y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) = 1$ and the point lies exactly on the margin. Use one of these to compute b.
- Non-margin support vectors, $\alpha_n = C$: then $\beta_n = 0$ allows $\xi_n > 0$, and $y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) = 1 - \xi_n$. Every margin violator ($\xi_n > 0$) is of this type.

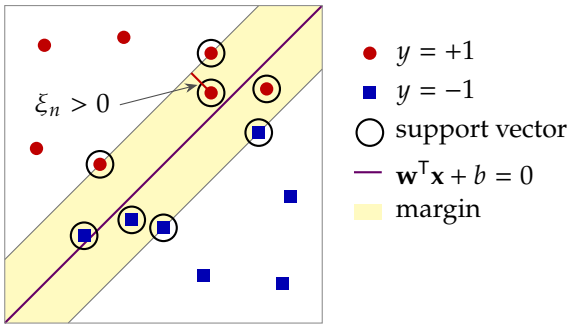


Figure 44: Schematic soft-margin solution. Circled points are support vectors: those on the margin boundaries have $0 < \alpha_n < C$; those inside the margin or on the wrong side have $\alpha_n = C$ and $\xi_n > 0$.

15.7 Two technical observations

1. *Hard margin on non-separable data.* If the data in $\mathcal{Z}$ are not linearly separable, the hard-margin primal has no feasible point, and the passage from primal to dual breaks down. A hard-margin solution returned by a solver must therefore be checked to separate the data.
2. *A constant coordinate in* $\mathcal{Z}$. If $\mathbf{z}$ has a constant coordinate with weight w_0, then w_0 and b play the same role, but only w_0 is penalized in $\mathbf{w}^\mathsf{T}\mathbf{w}$. The optimum puts the entire offset into b and sets $w_0 = 0$.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 15: Kernel Methods, Caltech, May 22, 2012. Lecture slides.

Lecture 16: Radial Basis Functions

Radial basis functions (RBFs) build a hypothesis from bumps centered at data points or at representative centers. The model connects to nearest neighbors, neural networks, kernel methods, and regularization.

16.1 The basic RBF model

In the basic model, each example $(\mathbf{x}_n, y_n) \in \mathcal{D}$ influences $h(\mathbf{x})$ through the distance $\|\mathbf{x} - \mathbf{x}_n\|$ alone; this dependence on distance is what makes it *radial*. The standard form uses a Gaussian basis function:

$$h(\mathbf{x}) = \sum_{n=1}^{N} w_n \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}_n\|^2\right), \qquad \gamma > 0.$$

Exact interpolation. To find $w_1, \ldots, w_N$ with $E_{\text{in}} = 0$, require $h(\mathbf{x}_n) = y_n$ for every n:

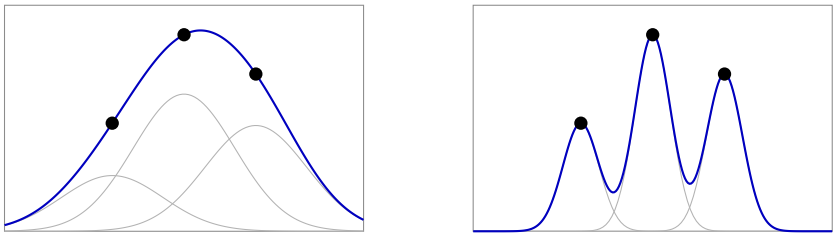
$$\sum_{m=1}^{N} w_m \exp\left(-\gamma \|\mathbf{x}_n - \mathbf{x}_m\|^2\right) = y_n, \qquad n = 1, \ldots, N.$$

These are N linear equations in N unknowns, $\Phi \mathbf{w} = \mathbf{y}$, where $\Phi_{nm} = \exp(-\gamma \|\mathbf{x}_n - \mathbf{x}_m\|^2)$. If Φ is invertible,

$$\boxed{\mathbf{w} = \Phi^{-1}\mathbf{y},}$$

which interpolates the data exactly.

The width parameter γ controls how the bumps combine. With small γ the bumps are wide and overlap, giving a smooth interpolant. With large γ they are narrow, so h still passes through every data point but falls toward zero between them.



small γ ($\gamma = 1$) large γ ($\gamma = 8$)

Figure 45: Exact interpolation of three illustrative points (dots) for two values of γ. Gray curves are the weighted Gaussians $w_n \exp(-\gamma(x - x_n)^2)$; blue is their sum h.

Classification. For $y = \pm 1$, use $h(\mathbf{x}) = \text{sign}(s(\mathbf{x}))$ with the signal

$$s(\mathbf{x}) = \sum_{n=1}^{N} w_n \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}_n\|^2\right),$$

and learn $\mathbf{w}$ as in linear regression for classification: minimize $\sum_n \left(s(\mathbf{x}_n) - y_n\right)^2$ on $\mathcal{D}$, then take the sign.

16.2 Relationship to nearest neighbors

The NEAREST-NEIGHBOR method adopts the y value of the closest training point, so its regions are the cells of points closest to each $\mathbf{x}_n$. A basis function has a similar effect: a cylinder (constant within a radius of $\mathbf{x}_n$, zero outside) copies y_n to nearby inputs, and the Gaussian is a smooth version of the same idea.

16.3 RBF with K centers

The basic model has N parameters $w_1, \ldots, w_N$ for N data points, which is too many for good generalization. Instead use $K \ll N$ CENTERS $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ in place of $\mathbf{x}_1, \ldots, \mathbf{x}_N$:

$$h(\mathbf{x}) = \sum_{k=1}^{K} w_k \exp\left(-\gamma \|\mathbf{x} - \boldsymbol{\mu}_k\|^2\right).$$

This requires two choices: the centers $\boldsymbol{\mu}_k$ and the weights w_k.

Choosing the centers. Place each center to minimize the distance between $\mathbf{x}_n$ and its closest center. This is K-MEANS CLUSTERING: split $\mathbf{x}_1, \ldots, \mathbf{x}_N$ into clusters $S_1, \ldots, S_K$ and minimize

$$\sum_{k=1}^{K} \sum_{\mathbf{x}_n \in S_k} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2.$$

The problem uses only the inputs, so it is unsupervised learning; it is also NP-hard. LLOYD'S ALGORITHM minimizes it iteratively with respect to the $\boldsymbol{\mu}_k$ and the S_k in turn.

Each iteration alternates two steps, with ties broken so that each point joins exactly one cluster. For fixed centers, assigning each point to its nearest center minimizes the objective; for fixed clusters, the mean minimizes $\sum_{\mathbf{x}_n \in S_k} \|\mathbf{x}_n - \boldsymbol{\mu}\|^2$ over $\boldsymbol{\mu}$. Neither step increases the objective, and there are finitely many partitions, so the algorithm converges. It reaches a *local* minimum, which depends on the initial centers.

Algorithm 16.1: Lloyd's algorithm

Input: Inputs $\mathbf{x}_1, \dots, \mathbf{x}_N$ (no labels); number of centers K
Output: Centers $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$

Initialize the centers $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$
repeat
  $S_k \leftarrow \{\mathbf{x}_n : \|\mathbf{x}_n - \boldsymbol{\mu}_k\| \le \|\mathbf{x}_n - \boldsymbol{\mu}_\ell\| \text{ for all } \ell\}$ for $k = 1, \dots, K$
  $\boldsymbol{\mu}_k \leftarrow \frac{1}{|S_k|} \sum_{\mathbf{x}_n \in S_k} \mathbf{x}_n$ for $k = 1, \dots, K$
until the clusters no longer change
return $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$

The resulting centers differ from support vectors. Support vectors are training points near the decision boundary, selected using the labels. RBF centers are cluster representatives spread across the input distribution, chosen without the labels and generally not data points.

Choosing the weights. With the centers fixed, ask for

$$\sum_{k=1}^{K} w_k \exp\bigl(-\gamma \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2\bigr) \approx y_n, \qquad n = 1, \dots, N.$$

These are N equations in $K < N$ unknowns, $\Phi \mathbf{w} \approx \mathbf{y}$ with the $N \times K$ matrix $\Phi_{nk} = \exp(-\gamma \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2)$. If $\Phi^{\mathsf{T}}\Phi$ is invertible, the least-squares solution is the pseudo-inverse

$$\boxed{\mathbf{w} = (\Phi^{\mathsf{T}}\Phi)^{-1}\Phi^{\mathsf{T}}\mathbf{y}.}$$

16.4 RBF networks and neural networks

An RBF NETWORK uses the features $\exp(-\gamma \|\mathbf{x} - \boldsymbol{\mu}_k\|^2)$ and combines them linearly; a bias term (b or w_0) is often added. Because the centers are computed from $\mathcal{D}$, the nonlinear transform depends on the data, so this is no longer a linear model with a fixed transform.

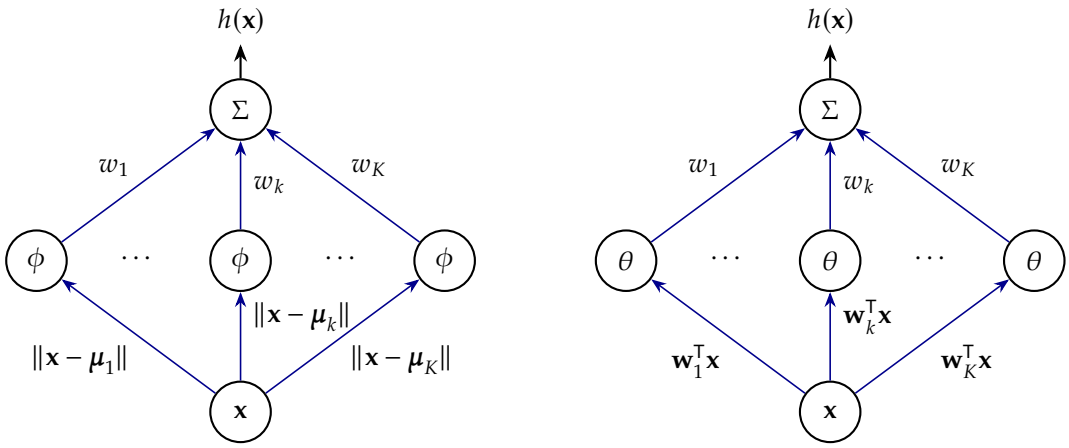


Figure 46: An RBF network and a two-layer neural network. The output node forms the weighted sum. RBF hidden units respond to the distance $\|\mathbf{x} - \boldsymbol{\mu}_k\|$; neural units apply θ to the signal $\mathbf{w}_k^\mathsf{T}\mathbf{x}$.

The architectures match. The difference is the hidden unit: a Gaussian of $\|\mathbf{x} - \boldsymbol{\mu}_k\|$ decays away from its center, while $\theta(\mathbf{w}_k^\mathsf{T}\mathbf{x})$ depends on the input through a projection, regardless of how far $\mathbf{x}$ lies from any reference point.

Choosing γ. Treat γ as a parameter to be learned in $h(\mathbf{x}) = \sum_k w_k \exp(-\gamma\|\mathbf{x} - \boldsymbol{\mu}_k\|^2)$. An iterative approach, similar to the EM algorithm for mixtures of Gaussians, alternates:

1. fix γ and solve for $w_1, \ldots, w_K$ by the pseudo-inverse;
2. fix $w_1, \ldots, w_K$ and minimize the error with respect to γ.

Each center $\boldsymbol{\mu}_k$ can also have its own width γ_k.

16.5 RBF and kernel methods

The SVM with the Gaussian kernel implements

$$\operatorname{sign}\left(\sum_{\alpha_n > 0} \alpha_n y_n \exp\left(-\gamma\|\mathbf{x} - \mathbf{x}_n\|^2\right) + b\right),$$

while a straight RBF classifier implements

$$\operatorname{sign}\left(\sum_{k=1}^{K} w_k \exp\left(-\gamma\|\mathbf{x} - \boldsymbol{\mu}_k\|^2\right) + b\right).$$

The functional forms are the same. In the SVM, the centers are the support vectors and the coefficients come from maximizing the margin, a supervised criterion. In the RBF network, the centers come from unsupervised clustering and the weights from the pseudo-inverse. On the same data, the two methods therefore produce different boundaries.

16.6 RBF and regularization

RBFs can also be derived purely from regularization. For scalar inputs, minimize

$$\sum_{n=1}^{N}\big(h(x_n)-y_n\big)^2+\lambda\sum_{k=0}^{\infty}a_k\int_{-\infty}^{\infty}\left(\frac{d^k h}{dx^k}\right)^2 dx.$$

The penalty charges every derivative of h, so the problem asks for the *smoothest interpolation* of the data. With suitable coefficients a_k, the minimizer is a combination of Gaussian radial basis functions centered at the data points.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 16: Radial Basis Functions, Caltech, May 24, 2012. Lecture slides.

Lecture 17: Three Learning Principles

Three principles govern how learning should be practiced and how its results should be interpreted: Occam's razor, sampling bias, and data snooping.

17.1 Occam's razor

A recurring theme is a preference for simple hypotheses: an explanation of the data should be made as simple as possible, but no simpler. The principle attributed to William of Occam is stated as follows.

Note. **Occam's razor.** The simplest model that fits the data is also the most plausible.

This raises two questions: what it means for a model to be simple, and how we know that simpler is better.

What "simple" means. Complexity can be measured for a single hypothesis or for a hypothesis set.

- *Complexity of h*: minimum description length (MDL), the order of a polynomial.
- *Complexity of $\mathcal{H}$*: entropy, VC dimension.

Intuition judges simplicity in terms of h; the proofs use simplicity in terms of $\mathcal{H}$. Counting links the two: if ℓ bits specify h, then h is one of 2^ℓ elements of a set $\mathcal{H}$. A simple h belongs to a small set. The link extends to real-valued parameters: a 17th-order polynomial is a complex h and one of many. Exceptions exist: an SVM boundary can look complex while being determined by a few support vectors, making it one of few.

Why simpler is better. "Better" means better out-of-sample performance, not more elegance. The basic argument, which can be made into formal proofs under different idealized conditions, runs as follows. There are fewer simple hypotheses than complex ones: the growth function $m_{\mathcal{H}}(N)$ is small. A small $\mathcal{H}$ is therefore less likely to fit a given data set: of the 2^N ways to label N points, at most a fraction $m_{\mathcal{H}}(N)/2^N$ can be matched. Consequently, a fit by a simple model is more significant when it happens.

Example: *The football oracle.* A letter predicts the outcome of a game, and the call is correct. More letters arrive for five weeks, all correct. The sender then offers the next prediction for \$50. Should you pay?

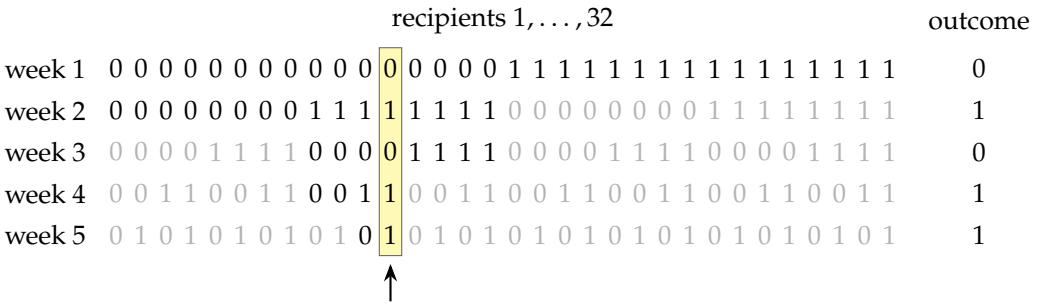


Figure 47: Each column lists the five predictions mailed to one recipient. Black predictions were mailed; gray ones were never sent because that recipient had already received a wrong call. Only the highlighted recipient sees five correct predictions.

No. The sender mailed all $2^5 = 32$ prediction sequences, half predicting each outcome, and each week wrote again only to those who had received a correct call: $32 + 16 + 8 + 4 + 2 = 62$ letters in total. From the recipient's view, a single hypothesis matched five outcomes, as if $m_{\mathcal{H}}(N) = 1$. In fact $m_{\mathcal{H}}(N) = 2^N$: some recipient was guaranteed a perfect record, so the fit carries no information about the next game.

Example: *A fit that means nothing.* Is conductivity linear in temperature? Scientist A measures resistivity at two temperatures; scientist B measures it at three. Both find their points on a line. A line passes through *any* two points, so A's fit could not have failed and provides no evidence. B's third point could have fallen off the line, so B's fit is evidence for linearity. Data provide evidence for a hypothesis only if they could have falsified it.

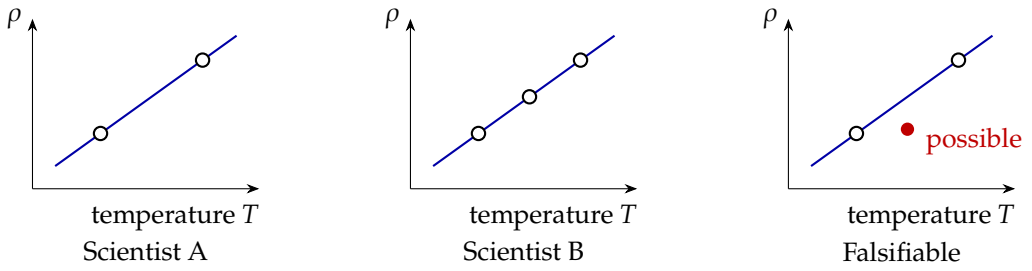


Figure 48: Schematic measurements of resistivity ρ. The right panel shows an outcome (red) that would have contradicted the linear hypothesis had it occurred.

17.2 Sampling bias

Example: *The 1948 presidential election.* Truman ran against Dewey in a close election. A newspaper conducted a phone poll of how people voted; Dewey won the poll decisively, and the newspaper declared Dewey the winner. Truman won the election.

The failure was not bad luck. The Hoeffding-type guarantee

$$\mathbb{P}\left[|E_{\text{in}} - E_{\text{out}}| > \epsilon\right] \leq \delta$$

was not at fault, since a larger poll would not have helped. In 1948, phones were expensive, so the poll sampled a population different from the voters.

Note. **Sampling bias.** If the data is sampled in a biased way, learning will produce a similarly biased outcome.

For example, a trading system trained on data from a normal period of the market is tested by live trading in the real market, which includes periods that are not normal.

Matching the distributions. There are methods that adjust the training sample to match the testing distribution. They cannot work if some region has $P = 0$ in training but $P > 0$ in testing: no reweighting creates data where none were sampled.

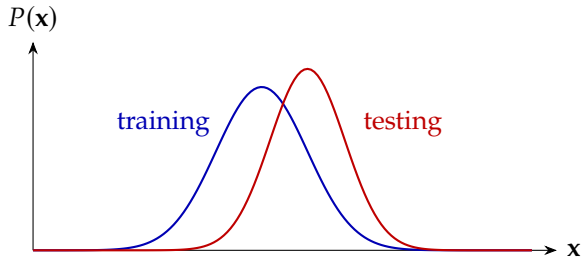


Figure 49: Schematic training and testing input distributions that differ.

Example: *Credit approval.* Historical records of customers provide the inputs (information from the credit application: age, gender, annual salary, years in residence, years in job, current debt, and so on) and the target (whether the customer was profitable for the bank). The records exist only for applicants who were approved, so the training data come from a different distribution than the applicants on whom the system will be used.

17.3 Data snooping

Note. **Data snooping.** If a data set has affected any step in the learning process, its ability to assess the outcome has been compromised.

This is the most common trap for practitioners, and there are many ways to slip into it.

Looking at the data. Recall nonlinear transforms. The full second-order transform is $\mathbf{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$. After looking at data whose classes separate along a circle, one might choose $\mathbf{z} = (1, x_1^2, x_2^2)$ or even $\mathbf{z} = (1, x_1^2 + x_2^2)$. The smaller model was chosen by examining $\mathcal{D}$, so the learning process effectively searched the larger space, and the generalization guarantee must be charged for it. Snooping concerns $\mathcal{D}$ itself; choosing a model from other information, such as

knowledge of the problem obtained before seeing the data, is legitimate.

Example: *Financial forecasting.* Predict the daily change Δr_0 of the US dollar versus the British pound from the previous twenty changes $\Delta r_{-20}, \Delta r_{-19}, \ldots, \Delta r_{-1}$. The data were normalized, then split randomly into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$; the model was trained on $\mathcal{D}_{\text{train}}$ only and g was tested on $\mathcal{D}_{\text{test}}$. Over 500 test days, the cumulative profit rose to roughly 20 to 30 percent. The snooping was in the normalization: its statistics were computed from all the data, including the test set. With the normalization computed from the training data alone, the same system showed a loss.

Reuse of a data set. Trying one model after another on the same data set will eventually "succeed": if you torture the data long enough, it will confess. The relevant complexity is the VC dimension of the *total* learning model, every hypothesis set that was tried, and this may include what *others* tried on the same data, such as earlier published work on a benchmark. The key problem is matching a *particular* data set.

There are two remedies.

1. *Avoid* data snooping by strict discipline, for example by keeping the test set untouched until the end.
2. *Account for* data snooping by assessing how much the data have been contaminated.

Example: *Bias via snooping.* To test the long-term performance of "buy and hold" in stocks, use 50 years of data for all companies currently traded in the S&P 500, and assume you strictly followed buy and hold. The result is a great profit. It is spurious: selecting the companies by their *current* membership uses information from the end of the period and excludes companies that failed along the way. Snooping has caused a sampling bias.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 17: Three Learning Principles, Caltech, May 29, 2012. Lecture slides.

Lecture 18: Epilogue

The final lecture places the course within machine learning as a whole, then outlines two topics beyond it: Bayesian learning and aggregation.

18.1 The map of machine learning

Machine learning presents a large number of models, methods, and concepts. They can be organized into theory, techniques, and paradigms; techniques divide further into models and methods.

Theory	**Techniques**		**Paradigms**
	Models	*Methods*	
VC	linear	regularization	supervised
bias-variance	neural networks	validation	unsupervised
complexity	SVM	aggregation	reinforcement
Bayesian	nearest neighbors	input processing	active
	RBF		online
	Gaussian processes		
	SVD		
	graphical models		

18.2 Bayesian learning

The probabilistic approach of the course already treats the input distribution $P(\mathbf{x})$ and the target distribution $P(y \mid \mathbf{x})$ as unknown probability distributions. Bayesian learning extends the probabilistic role to all components, including the identity of the target function.

The likelihood $P(\mathcal{D} \mid h = f)$ can decide which h to choose. Bayesian learning asks instead for $P(h = f \mid \mathcal{D})$, which requires an additional probability distribution. By Bayes' rule,

$$P(h = f \mid \mathcal{D}) = \frac{P(\mathcal{D} \mid h = f)\, P(h = f)}{P(\mathcal{D})} \propto P(\mathcal{D} \mid h = f)\, P(h = f).$$

Here $P(h = f)$ is the prior and $P(h = f \mid \mathcal{D})$ is the posterior. Given the prior, we have the full distribution.

Example: *A prior for the perceptron.* A perceptron h is determined by $\mathbf{w} = (w_0, w_1, \ldots, w_d)$. A possible prior on $\mathbf{w}$ makes each w_i independent and uniform over $[-1, 1]$. This determines the prior $P(h = f)$ over hypotheses. Given $\mathcal{D}$, we can compute $P(\mathcal{D} \mid h = f)$, and together they give $P(h = f \mid \mathcal{D}) \propto P(h = f)\, P(\mathcal{D} \mid h = f)$.

A prior is an assumption. Even the most "neutral" prior asserts something. Saying that x is unknown in $[-1, 1]$ is not the same as saying that x is random and uniform over $[-1, 1]$. The true equivalent of an unknown x is a random x with the point mass $\delta(x - a)$, where a is unknown.

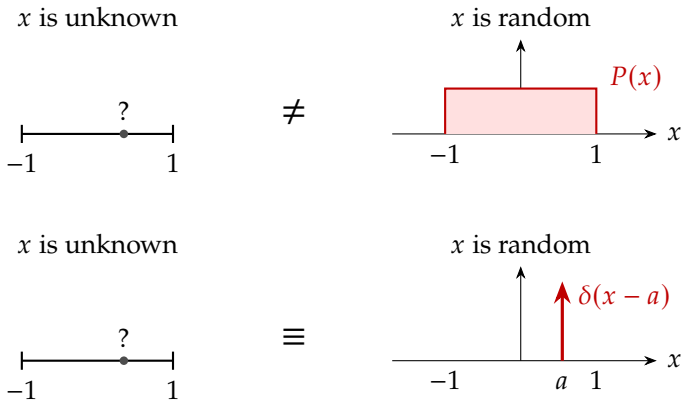


Figure 50: An unknown value is not a uniformly distributed one; it corresponds to a point mass at an unknown location a.

If we knew the prior, we could compute $P(h = f \mid \mathcal{D})$ for every $h \in \mathcal{H}$. We could then find the most probable h given the data, derive $\mathbb{E}[h(\mathbf{x})]$ for every $\mathbf{x}$, derive an error bar for every $\mathbf{x}$, and derive everything else in a principled way.

Bayesian learning is justified in two cases.

1. The prior is *valid*: it then trumps all other methods.
2. The prior is *irrelevant*: its effect on the result is negligible, and it serves only as a computational catalyst.

18.3 Aggregation methods

AGGREGATION combines different solutions $h_1, h_2, \ldots, h_T$ that were trained on $\mathcal{D}$. For regression, take an average; for classification, take a vote. Aggregation is also known as ENSEMBLE LEARNING and BOOSTING.

Aggregation differs from two-layer learning. In a two-layer model, all hidden units learn *jointly*, as parts of a single hypothesis. In aggregation, the solutions learn *independently* and are then combined.

There are two types of aggregation.

1. *After the fact*: combine existing solutions. Example: teams in the Netflix competition merged their solutions; this is BLENDING.

2. *Before the fact*: create solutions intended to be combined. Example: BAGGING trains each h_t on a resampling of $\mathcal{D}$.

Decorrelation by boosting. Create $h_1, \ldots, h_t, \ldots$ sequentially, making each h_t decorrelated with the previous ones. To do so, emphasize the points of $\mathcal{D}$ that were misclassified, and choose the weight of h_t in the combination based on $E_{\text{in}}(h_t)$.

Blending after the fact. For regression, combine $h_1, \ldots, h_T$ as

$$g(\mathbf{x}) = \sum_{t=1}^{T} \alpha_t h_t(\mathbf{x}).$$

A principled choice of the α_t minimizes the squared error on a separate *aggregation data set* $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_M, y_M)$. With the $M \times T$ matrix $H_{it} = h_t(\mathbf{x}_i)$, this is linear regression on the outputs of the h_t, solved by the pseudo-inverse:

$$\boldsymbol{\alpha} = (H^\mathsf{T} H)^{-1} H^\mathsf{T} \mathbf{y}$$

when $H^\mathsf{T} H$ is invertible. Some α_t can come out negative. The most valuable h_t in the blend is not necessarily the best on its own: uncorrelated h_t's help the blend.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 18: Epilogue, Caltech, May 31, 2012. Lecture slides.